

Data Engineering on Google Cloud

— Day 1 (of 2)

Iowa Department of Management, Department of IT · October 5, 2026 · Student slide pack

How to use this pack

- Slides follow the class agenda — each module section opens with a divider page showing its time slot.
- Hands-on activities (instructor-led demos you follow along):
<https://roitraining.github.io/gcp-demos/bigquery.html>
- Labs are in the Google Skills classroom — open the classroom, launch the lab listed for each block.

Agenda — Monday, October 5, 2026 (times CT)

Time	Block
09:00–09:25	Welcome, audience survey & the reframe (Module 0)
09:25–10:10	Module 1 — Data Engineering Tasks & Components
10:10–10:25	Hands-on console orientation (activity §3)
10:25–10:40	Break
10:40–11:10	Lab — Loading Data into BigQuery
11:10–11:50	Module 2 — Data Replication & Migration
11:50–12:00	Stream kickoff · Lab — Datastream (part 1: setup & start)
12:00–01:00	Lunch (Datastream replicating)
01:00–01:15	Lab — Datastream (part 2: validate backfill + changes)
01:15–01:45	Module 3 — Extract & Load pattern
01:45–02:30	Loading block — activities §24–§27 (console/CLI load, autodetect pitfall, CSV vs Parquet, LOAD DATA SQL)
02:30–02:45	Break
02:45–03:10	Module 4 — Extract, Load & Transform (ELT)
03:10–03:25	Dataform demo — your Silver→Gold tool
03:25–04:05	Lab — Create & Execute a SQL Workflow in Dataform
04:05–04:20	Modules 5+6 — ETL options & automation (compressed)
04:20–04:30	Close + scheduled-query demo

MODULE 0

Course Introduction

· 09:00–09:25 ·

all 24 slides

source deck: M0___T-DEGC-B___Course_Intro_ILT_Notes.pdf



Introduction to Data Engineering on Google Cloud

Name
Role

Hello and welcome to Introduction to Data Engineering on Google Cloud. Whether you already work in data engineering and want to learn how to be successful on Google Cloud or you are looking to progress in your career, this course will help you get started. Through a series of lectures, quizzes, and hands-on labs, you learn the fundamentals of data engineering on Google Cloud.

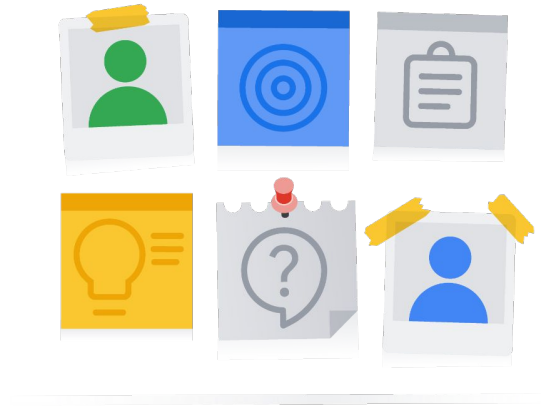
Introduction

Your instructor and you

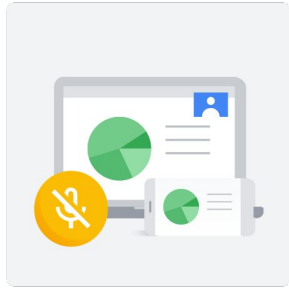
Background

Position

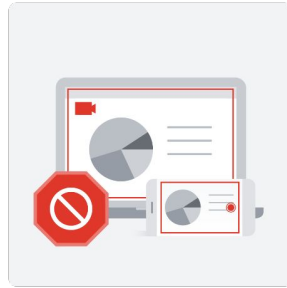
Organization



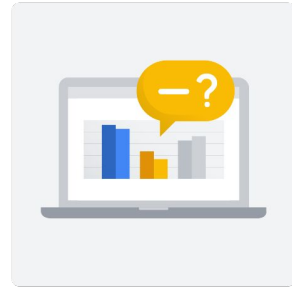
Etiquette



Mute microphone



No recording



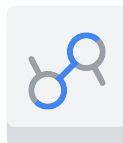
Ask questions

Target audience

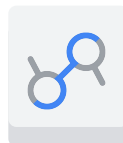
Data Engineer

Database Admin

System Admin



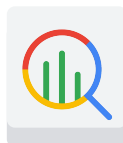
Dataflow



Managed
Apache Spark



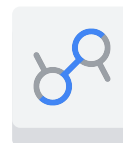
Managed
Airflow



BigQuery



Bigtable



Datastream

This course was designed for data engineers or anyone interested in preparing and storing data assets for further usage in their organization. This involves using tools such as, but not limited to Dataflow, Managed Service for Apache Spark, Managed Service for Apache Airflow, BigQuery, Bigtable, and Datastream.

Helpful knowledge



✓ Google Cloud basics

Helpful to be familiar with

✓ Querying data using SQL

✓ Python programming

✓ Data modelling and ETL

- Prior Google Cloud experience at the fundamental level using Cloud Shell and accessing products from the Google Cloud console.
- Basic proficiency with a common query language such as SQL.
- Experience with data modeling and ETL (extract, transform, load) activities.
- Experience developing applications using a common programming language such as Python.

Data Engineering on Google Cloud



Introduction to Data Engineering on Google Cloud

2

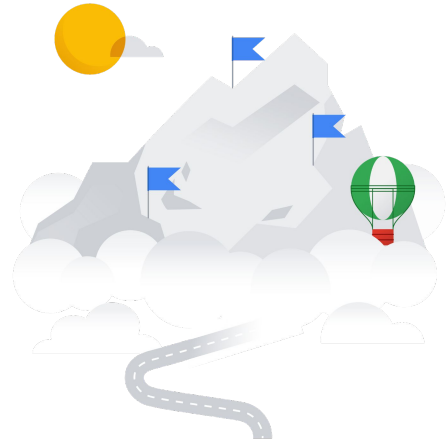
Build Data Lakes and Data Warehouses on Google Cloud

3

Build Batch Data Pipelines on Google Cloud

4

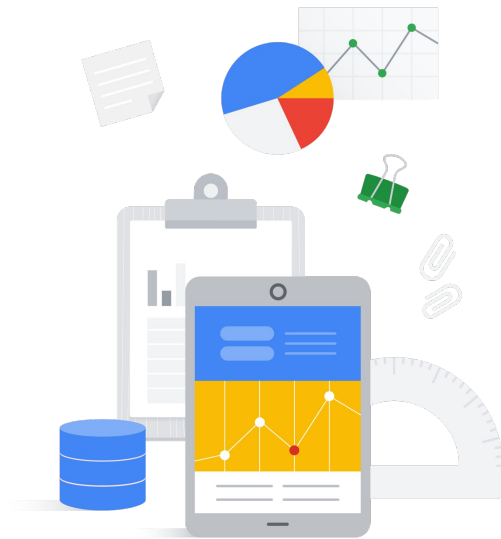
Build Streaming Data Pipelines on Google Cloud



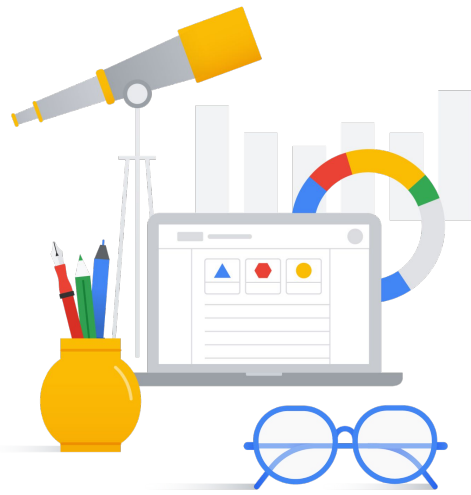
Introduction to Data Engineering on Google Cloud is the first course of Data Engineering on Google Cloud.

Learning objective #01

Understand the role of a data engineer.



In this course, you learn about the duties and responsibilities of a data engineer.



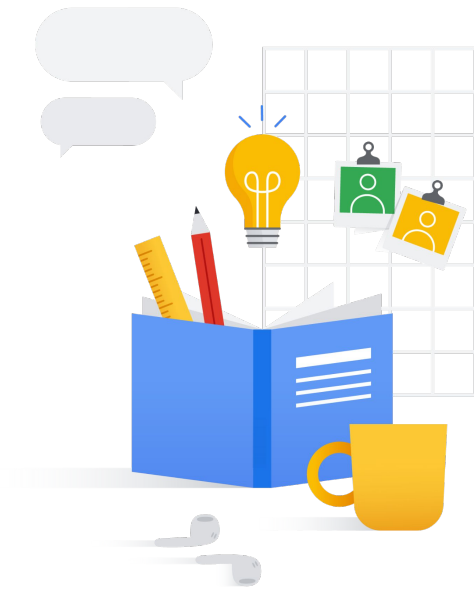
Learning objective #02

Identify data engineering tasks and core components used on Google Cloud.

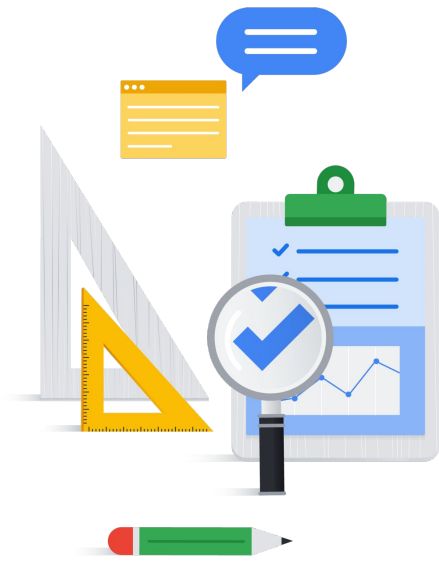
Identify data engineering tasks and core components used on Google Cloud to accomplish those tasks.

Learning objective #03

Understand how to create and deploy data pipelines of varying patterns on Google Cloud.



Understand how to create and deploy data pipelines of varying patterns on Google Cloud.



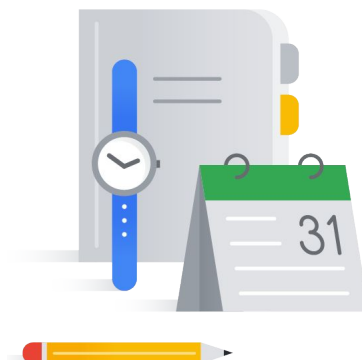
Learning objective #04

Identify and utilize various automation techniques on Google Cloud.

And identify and utilize various automation techniques on Google Cloud to complete data engineering tasks.

Agenda

- 01 Data Engineering tasks and components
- 02 Data replication and migration
- 03 The extract and load data pipeline pattern
- 04 The extract, load, and transform data pipeline pattern
- 05 The extract, transform, and load data pipeline pattern
- 06 Automation techniques



Introduction to Data Engineering on Google Cloud is divided into six modules designed to address the learning objectives.

First, we look at data engineering tasks and components on Google Cloud.

Next, we explore data replication and migration.

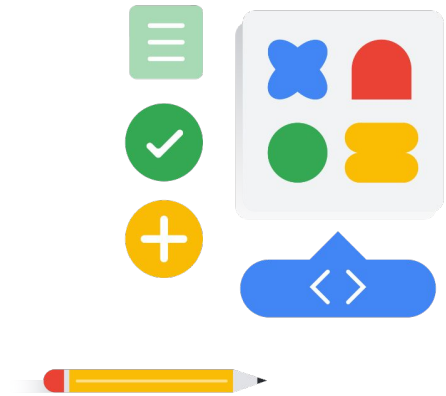
Then, we explore each of the three main data pipeline patterns: extract and load; extract, load, and transform; and extract, transform, and load.

We conclude the course by examining automation techniques important to a data engineer.

Lab environment

For each lab, Google Skills offers:

- ✓ A free set of resources for a fixed amount of time
- ✓ A clean environment with permissions



Open Google Skills

- 1 Open an incognito window (or private and/or anonymous window).
- 2 Go to the URL your instructor provides.
- 3 Sign In with existing account or **Join** with new account (with email you used to register for the course).
- 4 Launch the course from the **Dashboard** page.

Access issues

The process to open Google Skills can differ based on credentials used. Please reach out to your instructor if you have any access issues.

Up next

[Activities](#)[Classrooms](#)

SCHEDULED COURSE

Introduction to Data
Engineering on Google Cloud

[▶ Join](#)

View your labs

Do **NOT** launch a lab until instructed to do so!

Labs	Lecture Notes
	
	
	Lab Currently Disabled
	Lab Currently Disabled

← Lab completed

← To be completed

← Not yet available

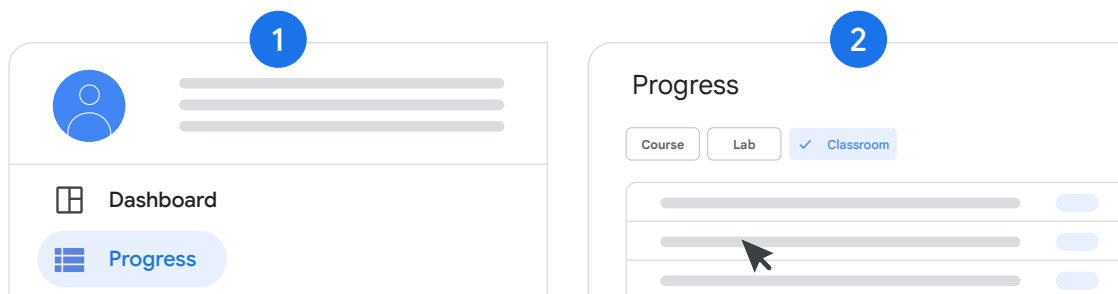
View lecture notes

Labs	Lecture Notes
01	⬇
02	⬇
03	⬇
04	⬇

You can download these as PDF files

End of class - Materials

Materials are available for 2 years



Click on your Profile and select **Progress**.

Select the class from the **Classroom** list.

You can view the course materials within Google Skills as follows:

1. Click on **My account** (top-right of the screen).
2. Select **Progress**.
3. Select the **Classroom** activity filter.
4. Select the class from the **Classroom** list.

Materials are available for 2 years following the completion of a course.

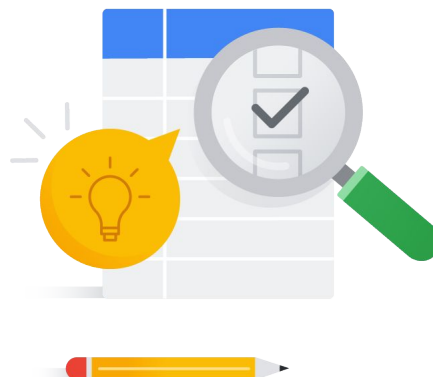
Lab: Loading Data into BigQuery

30:00

30 min

Learning objectives

- Load data into BigQuery from various sources.
- Load data into BigQuery using the CLI and the Google Cloud console.
- Use DDL to create tables.



Google Cloud

In this lab, you practice loading data into BigQuery. The primary objective of this lab is to load data into BigQuery using both the command-line interface and the Google Cloud console. You also experience loading several datasets into BigQuery and using the Data Description Language or DDL.

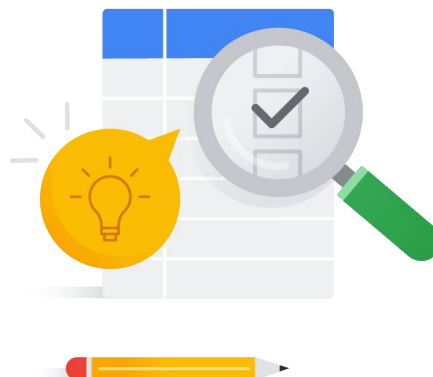
Lab URL: <https://www.skills.google/focuses/43259?parent=catalog>

Lab (optional): Datastream: PostgreSQL Replication to BigQuery

45:00 45 min

Learning objectives

- Prepare a Cloud SQL for PostgreSQL instance using the Google Cloud console.
- Import data into the Cloud SQL instance.
- Create a Datastream connection profile for the PostgreSQL database.
- Create a Datastream connection profile for the BigQuery destination.
- Create a Datastream stream and start replication.
- Validate that the existing data and changes are replicated correctly into BigQuery.



Google Cloud

In this lab, you use Datastream to replicate data from PostgreSQL to BigQuery. You prepare and load a Cloud SQL for PostgreSQL instance. You create Datastream connection profiles for the source and target. You then create a Datastream processing stream and start replication. Finally, you validate the replication in BigQuery.

Lab URL: <https://www.skills.google/focuses/53927?parent=catalog>

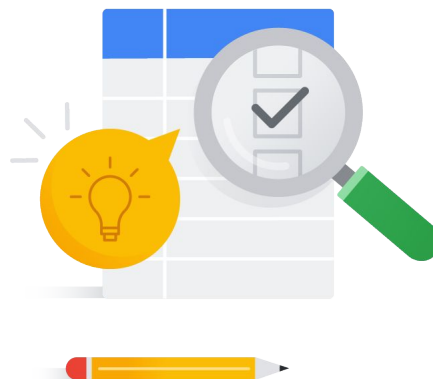
Lab: Lakehouse: Qwik Start

40:00

🕒 40 min

Learning objectives

- Create and view a connection resource.
- Set up access to a Cloud Storage data lake.
- Create a Lakehouse table.
- Query a Lakehouse table through BigQuery.
- Set up access control policies.
- Upgrade an external table to be a Lakehouse table.



Google Cloud

In this lab, you use Lakehouse for Apache Iceberg to connect to various external data sources.

You configure a connection resource and set up access to a Cloud Storage data lake.

You create and query a Lakehouse table and set up access control policies.

Finally, you upgrade an existing external table to be a Lakehouse table.

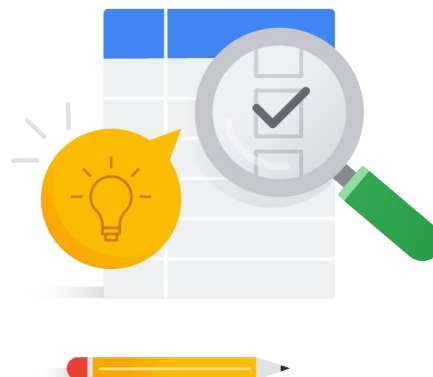
Lab URL: <https://www.skills.google/focuses/57211?parent=catalog>

Lab: Create and Execute a SQL Workflow in Dataform

🕒 30 min

Learning objectives

- Create a Dataform repository.
- Create and initialize a Dataform development workspace.
- Create and execute a SQL workflow.
- View execution logs in Dataform.



Google Cloud

In this lab, you use Dataform to create and execute a SQL workflow. First, you create a Dataform repository. Second, you create and initialize a Dataform development workspace. Then, you create and execute a SQL workflow. Finally, you view execution logs in Dataform to confirm completion.

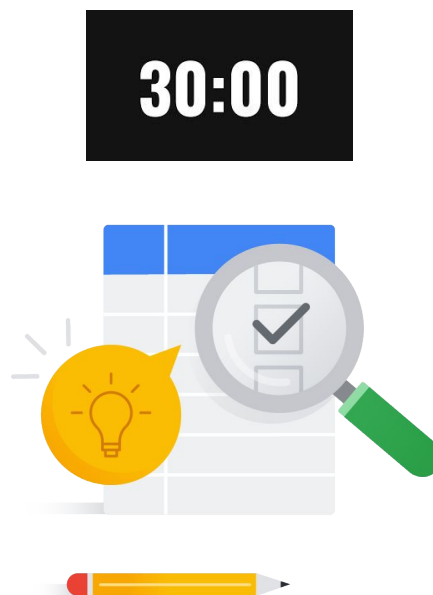
Lab URL: <https://www.skills.google/focuses/77087?parent=catalog>

Lab (optional): Use Serverless for Apache Spark to Load BigQuery

🕒 30 min

Learning objectives

- Configure the environment.
- Download lab assets.
- Configure and execute the Spark code.
- View data in BigQuery.



Google Cloud

In this lab, you use Serverless for Apache Spark to Load BigQuery.

First, you configure the environment.

Next you download lab assets.

You then configure and execute the Spark code.

Finally, you view the data in BigQuery.

reference: <https://www.skills.google/focuses/102966?parent=catalog>

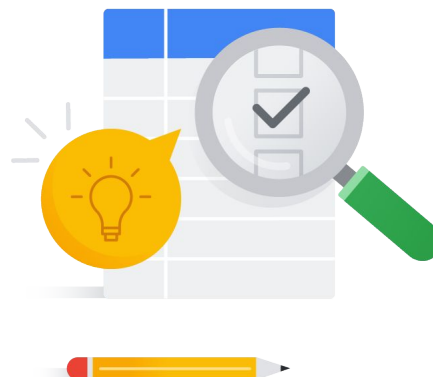
Lab: Creating a Streaming Data Pipeline for a Real-Time Dashboard with Dataflow

45:00

🕒 45 min

Learning objectives

- Create a Dataflow job from a template.
- Stream data via Dataflow pipeline into BigQuery.
- Monitor a Dataflow pipeline in BigQuery.
- Analyze results with SQL.
- Visualize key metrics in Looker Studio.



Google Cloud

In this lab, you create a streaming data pipeline for a real-time dashboard with Dataflow.

You create a Dataflow job from a template.

You then monitor a pipeline loading data into BigQuery.

After that, you examine the data loaded using SQL.

Finally, you visualize key metrics using Looker Studio.

reference: <https://www.skills.google/focuses/46207?parent=catalog>

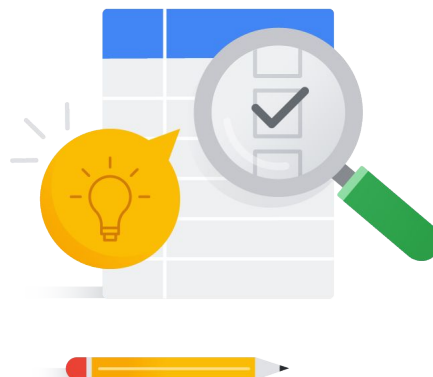
Lab (optional): Use Cloud Run Functions to Load BigQuery

🕒 30 min

30:00

Learning objectives

- Create a Cloud Run function.
- Deploy and test the Cloud Run function.
- View data in BigQuery and review Cloud Run function logs.



Google Cloud

In this lab, you create a Cloud Run function to Load BigQuery.

You create a Cloud Run function using the Google Cloud SDK.

You then deploy and test the Cloud Run function.

Finally, you view data in BigQuery and review Cloud Run function logs.

reference: <https://www.skills.google/focuses/102965?parent=catalog>



Let's get started!

MODULE 1

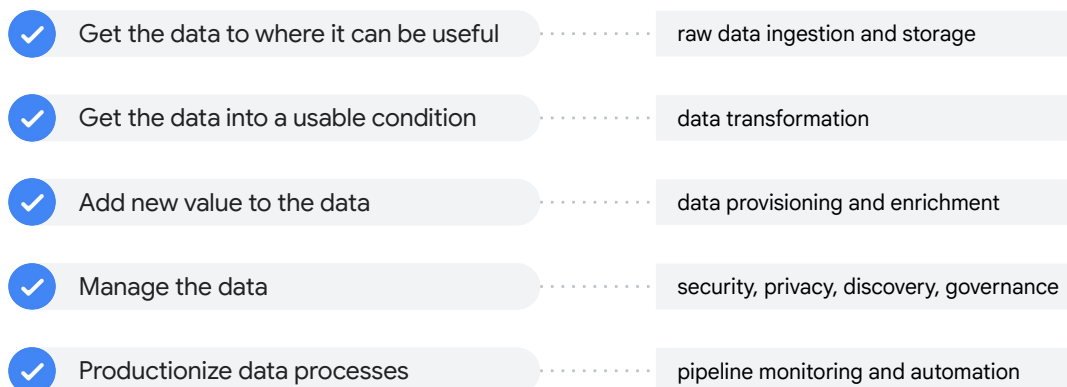
Data Engineering Tasks and Components

· 09:25–10:10 ·

slides 4–7, 9–11, 13, 15–18, 20–21, 22 (demo), 29 (lab intro)

source deck: [M1___T-DEGC-B___Data_Engineering_Tasks_and_Components_ILT_Notes.pdf](#)

A data engineer builds data pipelines to enable data-driven decisions



What does a data engineer do? At a basic level, a data engineer builds data pipelines.

Why does the data engineer build data pipelines? Because they want to get their data into a place, such as a dashboard or report or machine learning model, from where the business can make data-driven decisions.

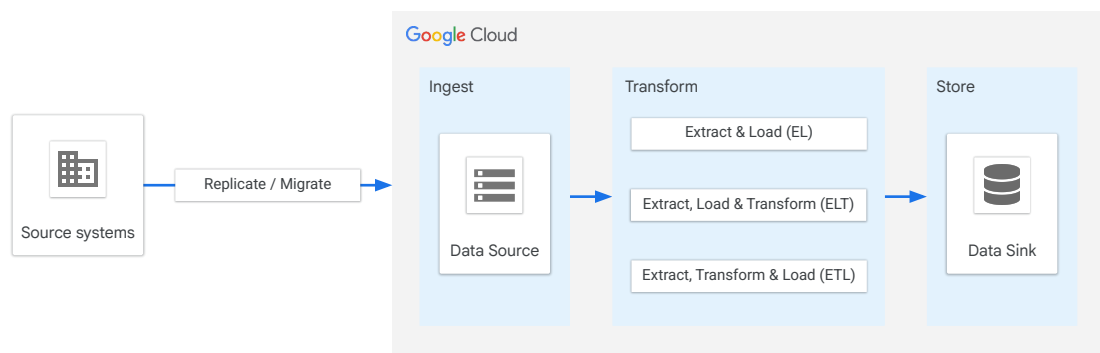
The data has to be in a usable condition so that someone can use this data to make decisions. Many times, the raw data is, by itself, not very useful.

Once data becomes useful, the data engineer will often apply updates or transformations to add new value to the data.

Of course, new data environments require data management practices to ensure currency and accuracy.

Finally, data engineers create processes and operations to move data usage into production settings.

Data engineering tasks evolve around ingesting, transforming, and storing data

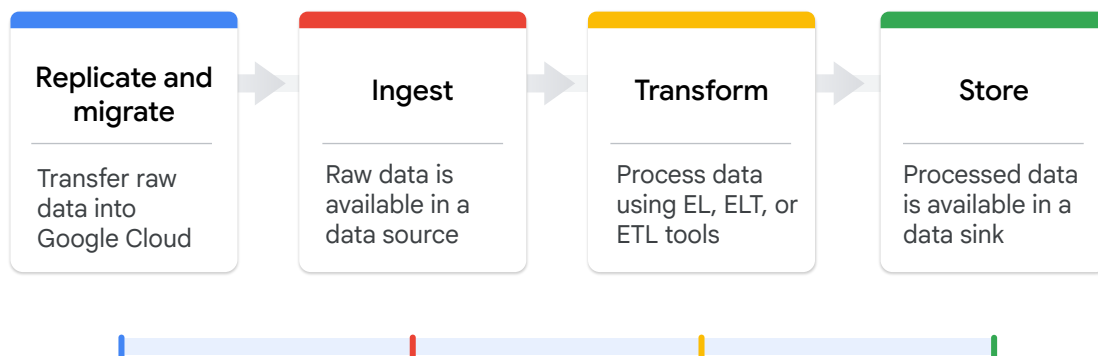


In the most basic sense, a data engineer moves data from data sources to data sinks via a transformation process.

As noted in the diagram there are generally three options or patterns to achieve those transformations.

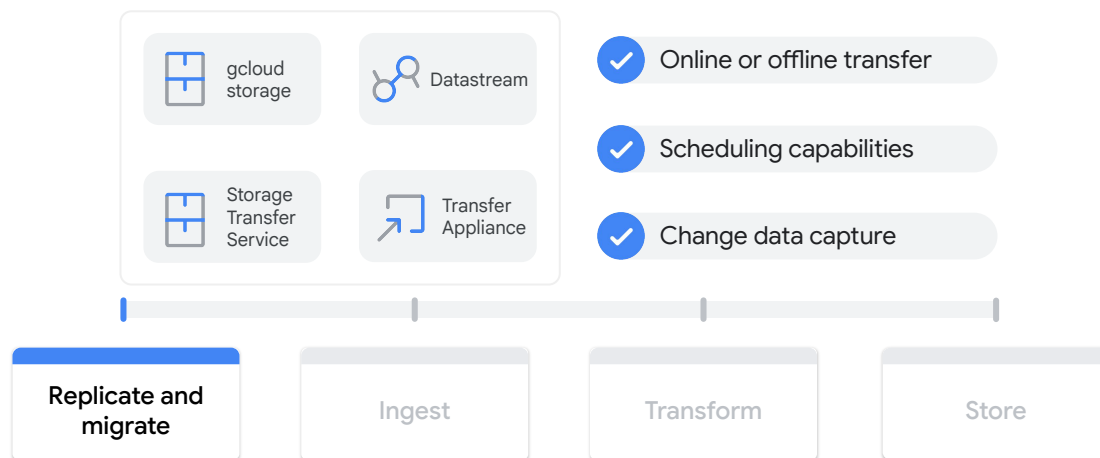
This course focuses in detail on those three patterns.

Data engineering tasks evolve around ingesting, transforming, and storing data



In the most basic sense, a data engineer moves data from data sources to data sinks in four stages: replicate and migrate, ingest, transform, and store.

Replication and migration services onboard your data into Google Cloud

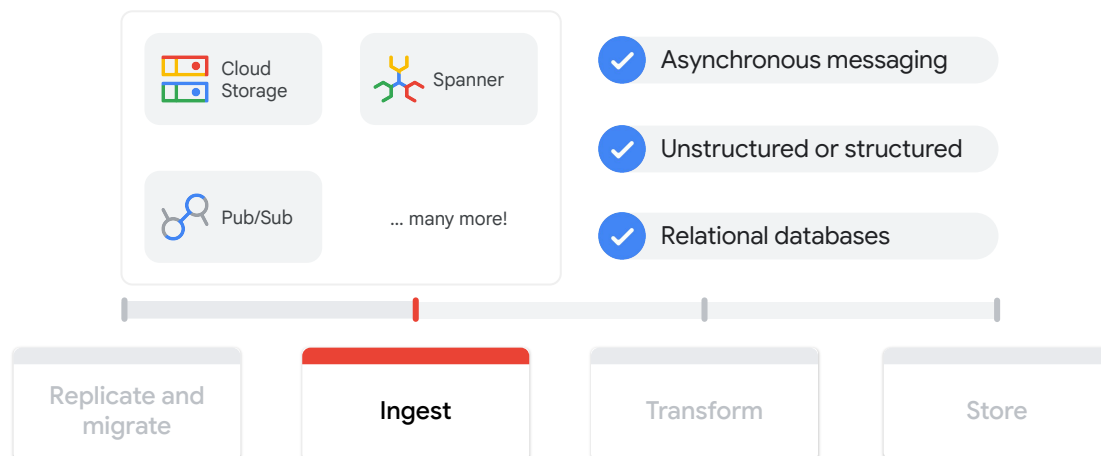


Google Cloud

The replicate and migrate stage of a data pipeline focuses on the tools and options to bring data from external or internal systems into Google Cloud for further refinement.

There are a wide variety of tools and options at your disposal. They will be covered in more detail throughout this course.

Data sources are the origin point of your raw data on Google Cloud

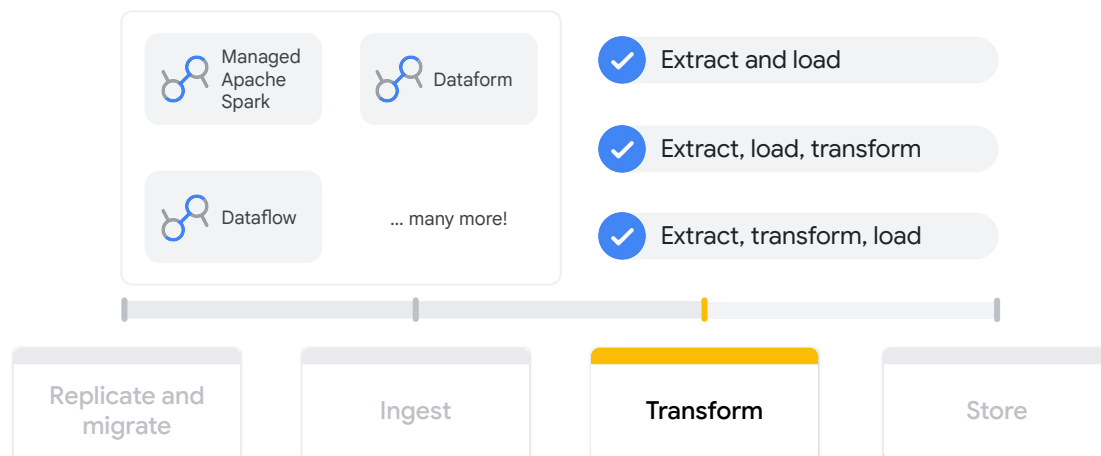


The ingest stage of a data pipeline is the point where data becomes a data source and is available for usage downstream.

Think of a data source as the starting point of your data journey. It is raw, unprocessed data waiting to be transformed into valuable insights. Any system, application, or platform that creates, stores, or shares data can be considered a data source.

Two examples of Google Cloud products used in the ingest phase are Cloud Storage, a data lake holding various types of data sources, and Pub/Sub, an asynchronous messaging system delivering data from external systems.

Transformation services add new value to your data



Google Cloud

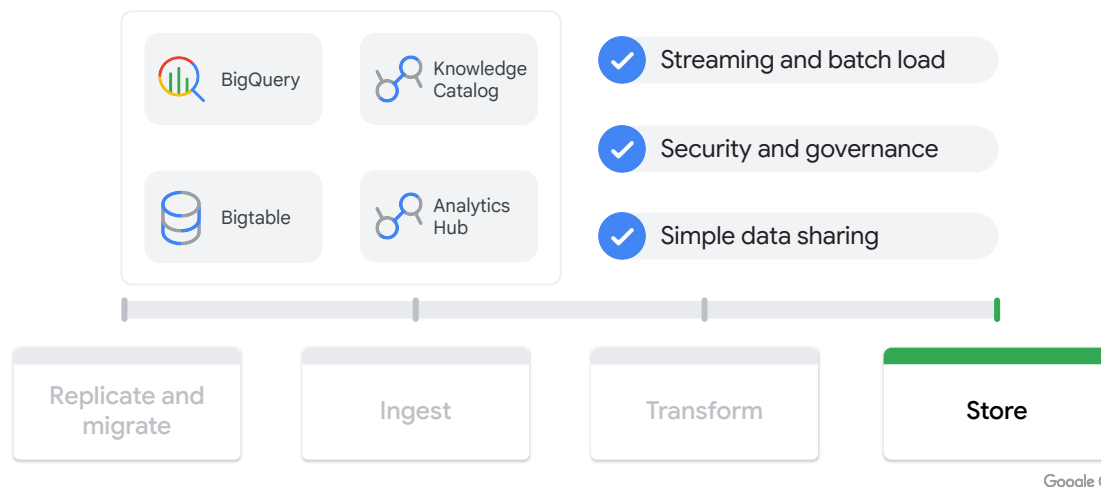
The transform stage of a data pipeline represents action taken on a data source to adjust, modify, join, or customize a data source so that it matches a specific downstream data or reporting requirement.

There are three main transformation patterns:

extract and load;
extract, load, and transform; and
extract, transform, and load.

We explore each of these patterns in their own modules later in the course.

Data sinks store your processed data on Google Cloud

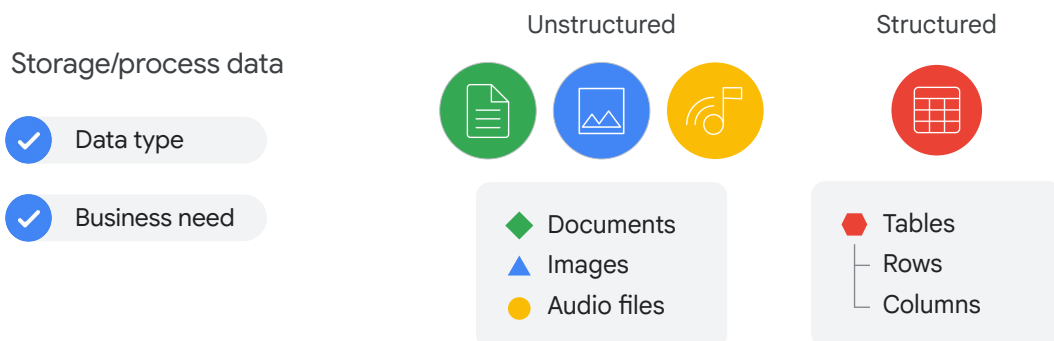


The store stage of a data pipeline represents the last step, when we deposit data in its final form.

A data sink is the final stop in the data journey. It's where processed and transformed data is stored for future use, analysis, and decision-making. Think of it as the reservoir at the end of the river, where valuable information is collected and readily available.

Two examples of Google Cloud products used in the store phase are BigQuery, a serverless data warehouse, and Bigtable, a highly scalable NoSQL database.

Data can have different formats

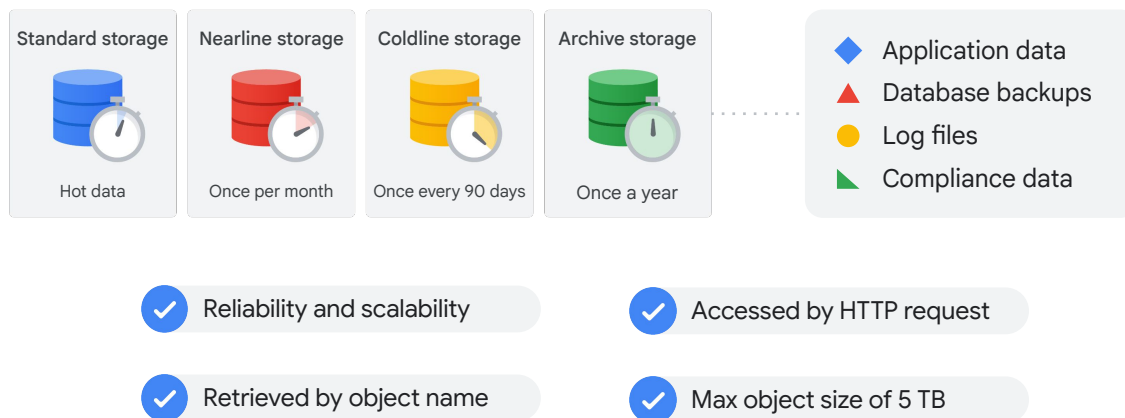


Data exists in two primary formats, unstructured and structured.

Unstructured data is information stored in a non-tabular form such as documents, images, and audio files. Unstructured data is usually suited for Cloud Storage, but BigQuery also offers the capability to store unstructured data via object tables.

There is also structured data, which represents information stored in tables, rows, and columns.

Cloud Storage holds your unstructured data



Google Cloud

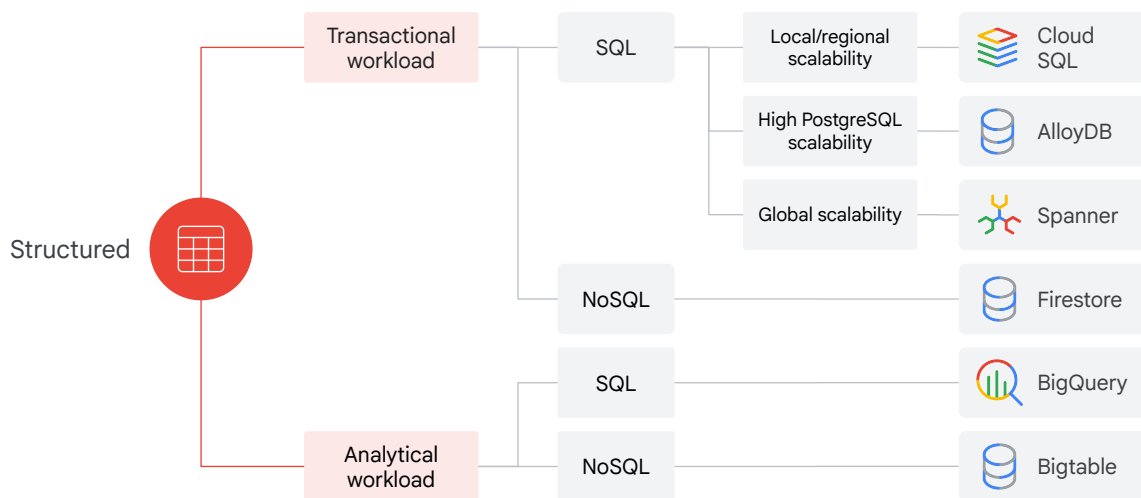
There are several key products on Google Cloud that are used by data engineers. One main product is Cloud Storage. Unstructured data is usually well-suited to be stored in Cloud Storage.

Within Cloud Storage, objects are accessed by using HTTP requests, including ranged GETs to retrieve portions of the data. The only key is the object name. There is object metadata but the object itself is treated as unstructured bytes. The scale of the system allows for serving large static content and accepting user-uploaded content including videos, photos, and files. Objects can be up to 5 Terabytes each.

Cloud Storage is built for availability, durability, scalability, and consistency. It's an ideal solution for hosting static websites and storing images, videos, objects and blobs, and any unstructured data.

Cloud Storage has four primary storage classes: standard storage, nearline storage, coldline storage, and archive storage. The classes are differentiated by the expected period of object access.

Options for storing structured data



Google Cloud

You have a full range of cost-effective storage services for structured data to choose from when developing with Google Cloud. No one size fits all, and your choice of storage and database solutions will depend on your application and workload.

Cloud SQL is Google Cloud's managed relational database service.

AlloyDB is a fully managed, high-performance PostgreSQL database service from Google Cloud.

Spanner is Google Cloud's fully managed relational database service that offers both strong consistency and horizontal scalability.

Firestore is a fast, fully managed, serverless, NoSQL document database built for automatic scaling, high performance, and ease of application development.

BigQuery is a fully managed, serverless enterprise data warehouse for analytics.

Bigtable is a high-performance NoSQL database service. Bigtable is built for fast key-value lookup and supports consistent sub 10 millisecond latency.

Data lake versus data warehouse

	Data lake	Data warehouse
Data format	Native (unstructured, semi-structured, structured)	Schema (structured or semi-structured)
Data type	Raw	Pre-processed and aggregated from multiple data sources
Purpose	Data science, applications, business decisions	Long-term business analysis
Dependencies	Tools/processes to enable data discovery, governance, security, metadata management	Standalone
Service	 Cloud Storage	 BigQuery

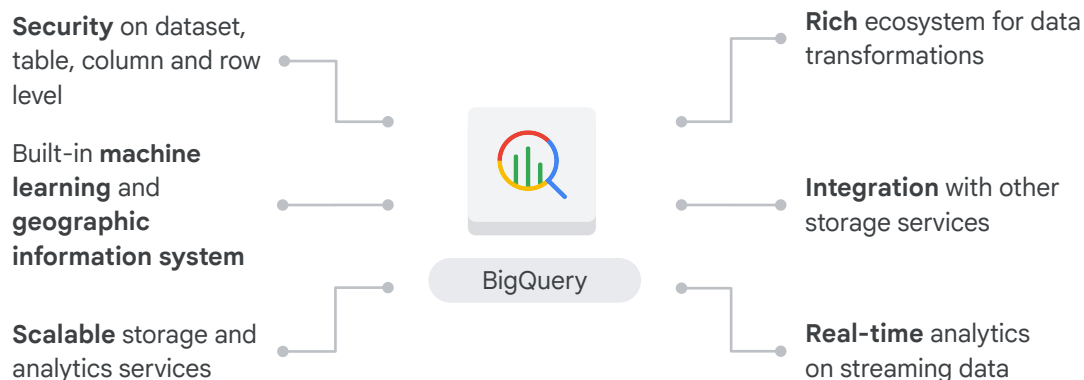
Google Cloud

The two key concepts in data engineering are that of the data lake and the data warehouse.

A data lake is a vast repository for storing raw, unprocessed data in various formats, including unstructured, semi-structured, and structured. It serves as a centralized storage solution for diverse data types, enabling flexible use cases like data science, applications, and business decision-making.

A data warehouse is a structured repository designed for storing pre-processed and aggregated data from multiple sources. Primarily used for long-term business analysis, it enables efficient querying and reporting for informed decision-making. Data warehouses often operate as standalone systems, independent of other data storage solutions.

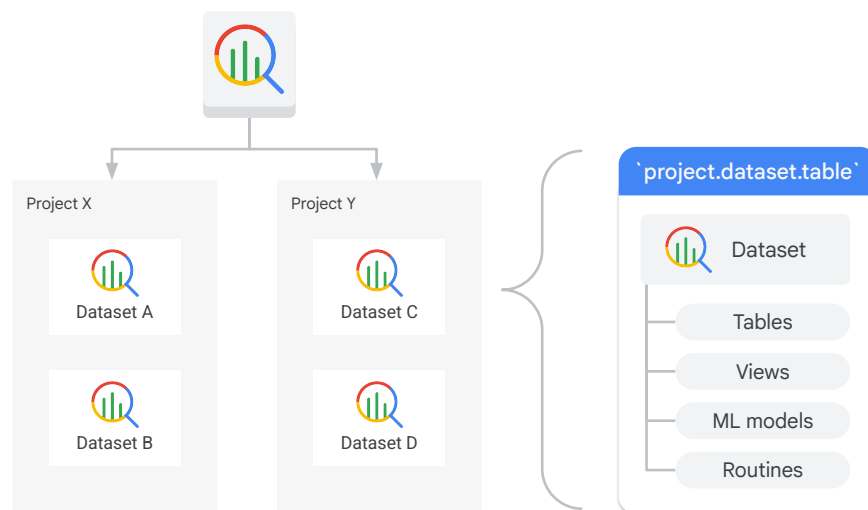
BigQuery is a serverless, fully managed data warehouse



BigQuery is a fully managed, serverless enterprise data warehouse for analytics. BigQuery has built-in features like machine learning, geospatial analysis, and business intelligence. BigQuery can scan terabytes in seconds, and petabytes in minutes.

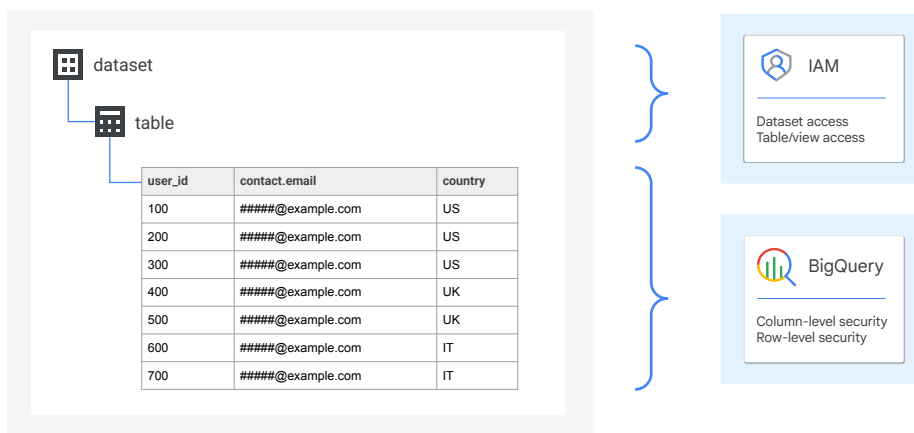
BigQuery is a great solution for online analytical processing, or OLAP, workloads for big data exploration and processing. BigQuery is also well-suited for reporting with business intelligence tools.

How BigQuery's resources are organized



BigQuery organizes data tables into units called datasets. These datasets are scoped to your Google Cloud project. When you reference a table from the command line in SQL queries or in code, you refer to it by using the construct `project.dataset.table`.

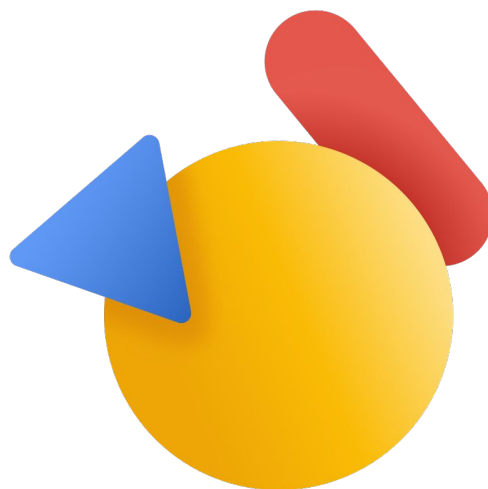
You can secure your BigQuery resources on multiple levels



Access control is through IAM and is at the dataset, table or view, or column level. In order to query data in a table or view, you need at least read permissions on the table or view.

Demo

Query terabytes of data in seconds

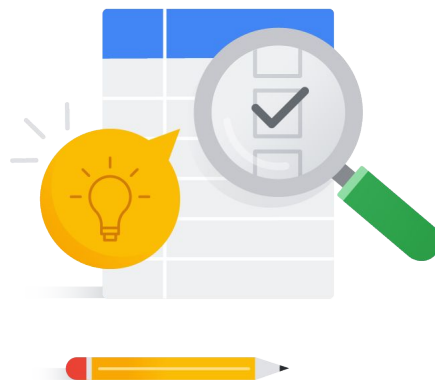


Lab: Loading Data into BigQuery

🕒 30 min

Learning objectives

- Load data into BigQuery from various sources.
- Load data into BigQuery using the CLI and the Google Cloud console.
- Use DDL to create tables.



Google Cloud

In this lab, you practice loading data into BigQuery. The primary objective of this lab is to load data into BigQuery using both the command-line interface and the Google Cloud console. You also experience loading several datasets into BigQuery and using the Data Description Language or DDL.

Lab URL: <https://www.skills.google/focuses/43259?parent=catalog>

MODULE 2

Data Replication and Migration

· 11:10-11:50 ·

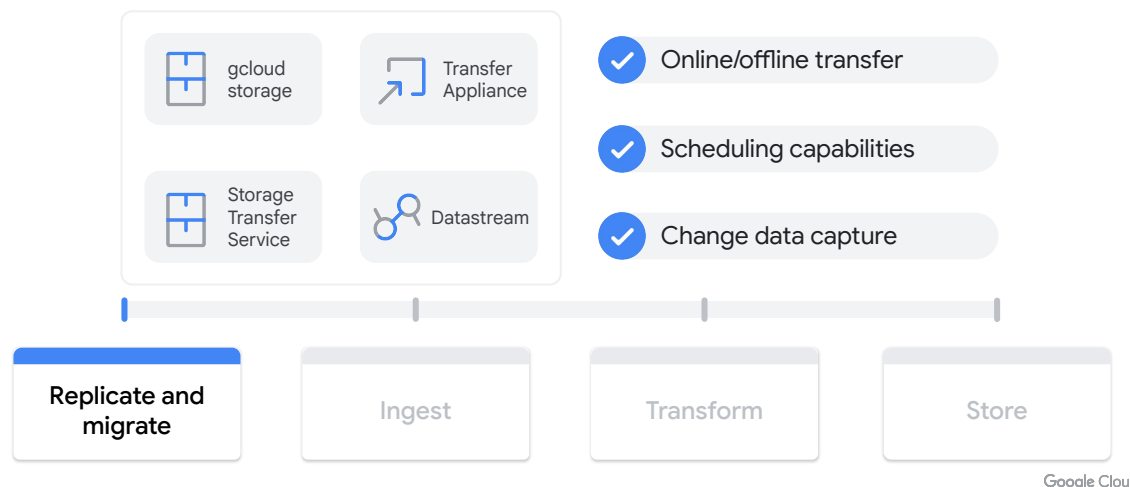
slides 1, 4-5, 7, 11, 14-16, 19-21

source deck: M2___T-DEGC-B___Data_Replication_and_Migration_ILT_Notes.pdf



Data replication and migration

Replication and migration services onboard your data into Google Cloud



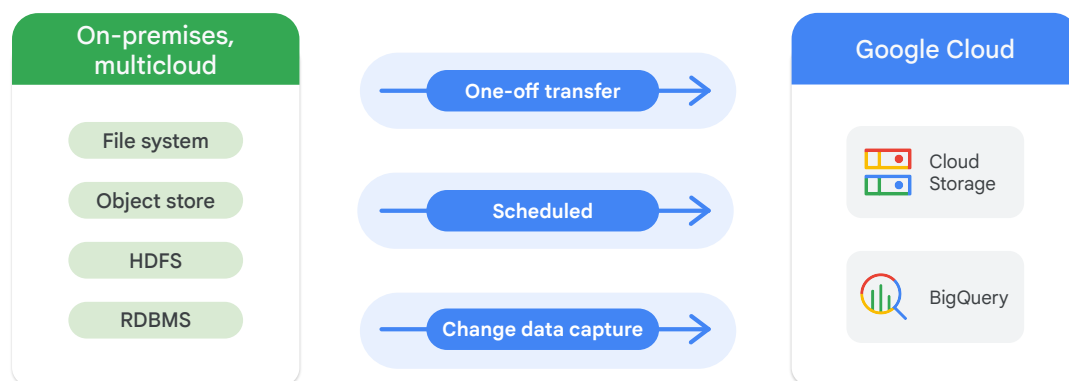
The replicate and migrate stage of a data pipeline focuses on the tools and options to bring data from external or internal systems into Google Cloud for further refinement.

Google Cloud provides a comprehensive suite of tools to migrate and replicate your data.

Start replicating and migrating data by using tools like the “gcloud storage” command, Transfer Appliance, Storage Transfer Service, or Datastream.

You can then transform the data as needed before finally storing it within Google Cloud.

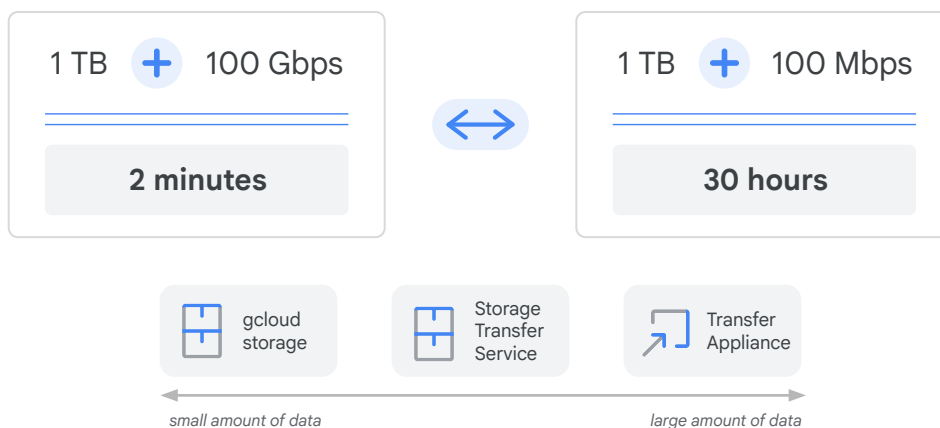
Common replication and migration scenarios for ingesting your data into Google Cloud



Data can originate from on-premises or multicloud environments, including file systems, object stores, HDFS, and Relational Databases.

Google Cloud offers options for one-off transfers, scheduled replications, and change data capture, ultimately landing data in Cloud Storage or BigQuery.

Choosing between migration options depends on your amount of data and network bandwidth



The ease of migrating data depends heavily on data size and network bandwidth.

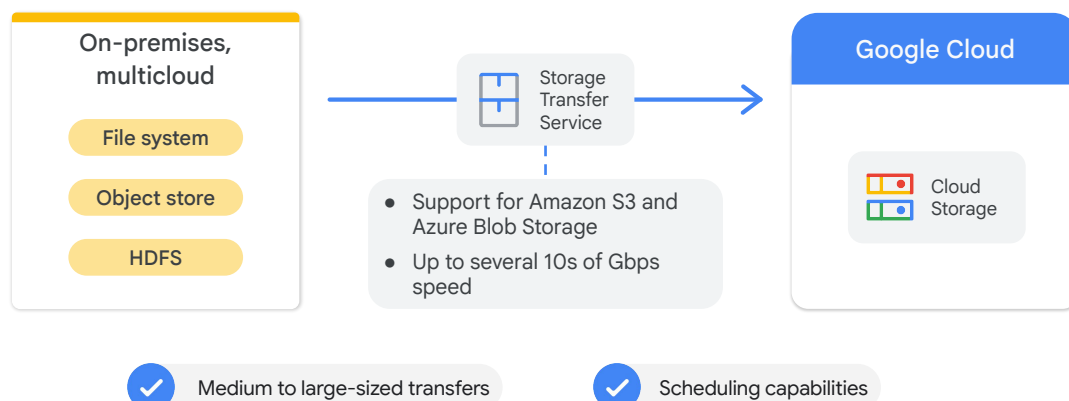
With 1 Terabyte of data, a 100 Gigabits per second network takes about 2 minutes to transfer, while the same size on a 100 Megabits per second network takes 30 hours.

The "gcloud storage" command or Storage Transfer Service are suitable for smaller datasets. For larger datasets, consider Transfer Appliance for faster offline transfer.

References:

https://cloud.google.com/architecture/migration-to-google-cloud-transferring-your-large-datasets#online_versus_offline_transfer

Move large amounts of data with Storage Transfer Service



To move larger datasets, consider using Storage Transfer Service.

Storage Transfer Service efficiently moves large datasets from on-premises, multicloud file systems, object stores (including Amazon S3 and Azure Blob Storage), and HDFS into Cloud Storage.

It boasts high transfer speeds (up to tens of Gigabytes per second) and supports scheduled transfers for convenient data migration.

References:

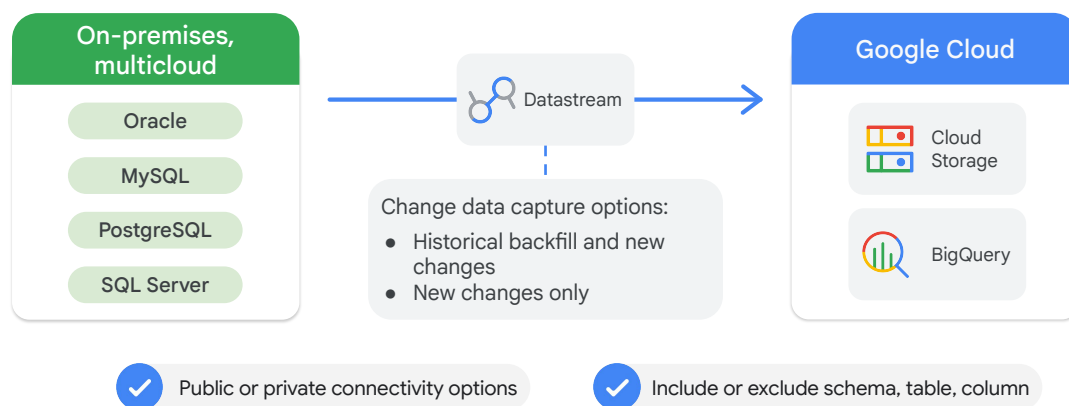
<https://cloud.google.com/storage-transfer/docs/sources-and-sinks>

<https://cloud.google.com/architecture/migration-to-google-cloud-transferring-your-large-datasets#storage-transfer-service-for-large-transfers-of-on-premises-data>

Storage transfer service scaling limitations (several 10s of Gbps):

<https://cloud.google.com/storage-transfer/docs/known-limitations-transfer#scaling>

Datastream continuously replicates your RDBMS into Google Cloud for analytics



Google Cloud

Datastream enables continuous replication of your on-premises or multicloud relational databases such as Oracle, MySQL, PostgreSQL, or SQL Server into Google Cloud.

Datastream offers change data capture options for historical backfill or allows you to just propagate new changes, with data landing in Cloud Storage or BigQuery for analytics.

You have flexibility in connectivity options and can selectively replicate data at the schema, table, or column level.

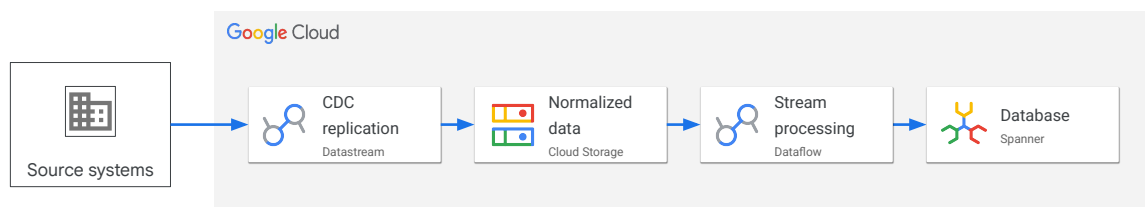
References:

<https://cloud.google.com/datastream/docs/sources>

<https://cloud.google.com/datastream/docs/network-connectivity-options>

Datastream use cases

- Analytics with database replication into BigQuery
- Analytics with custom data processing in Dataflow into BigQuery
- Data processing using an event-driven architecture
- **Example:** database replication and migration using Dataflow templates



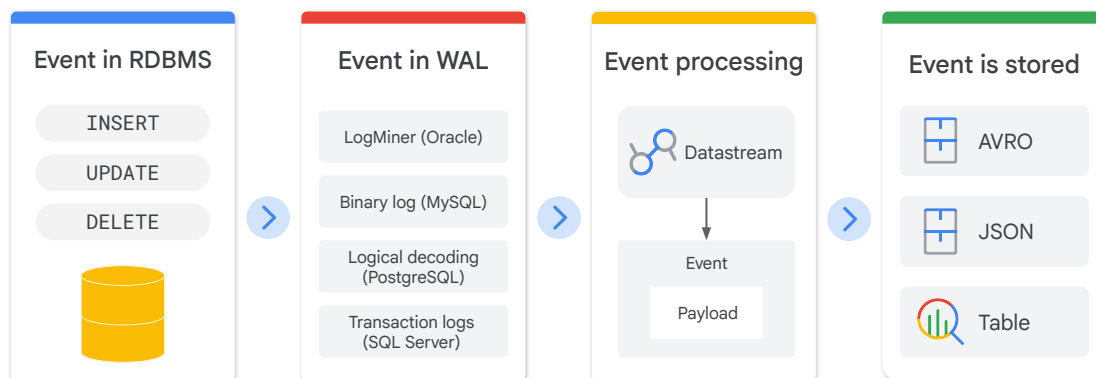
Google Cloud

Datastream enables real-time data replication from source systems for various use cases.

It supports direct replication into BigQuery for analytics, allows custom data processing in Dataflow before loading into BigQuery, and facilitates event-driven architectures.

Additionally, Datastream can be used with Dataflow templates for seamless database replication and migration tasks, making it a versatile tool for integrating data into Google Cloud.

Datastream uses the database's write-ahead log (WAL) to process change events

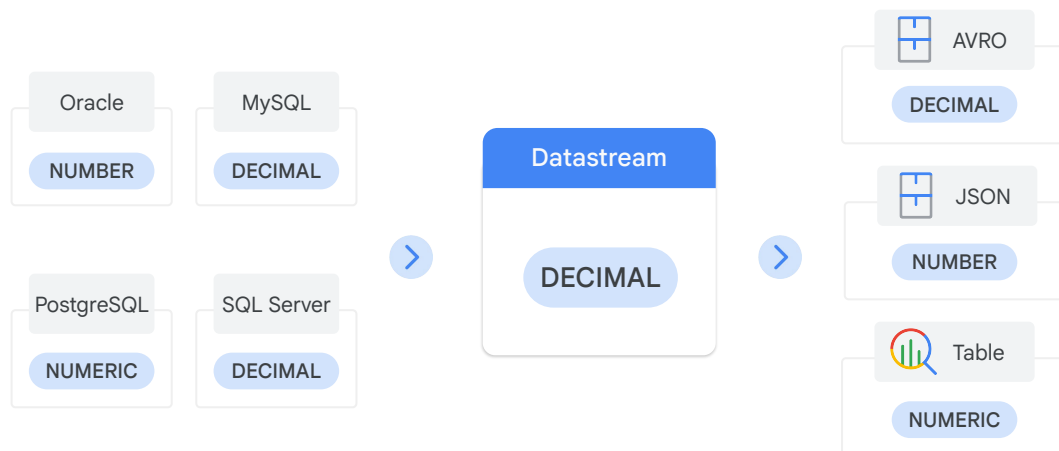


Datastream taps into the source database's write-ahead log (WAL) to capture and process changes for propagation downstream.

Datastream supports reading the logging mechanisms for the specific source database such as LogMiner for Oracle, binary log for MySQL, PostgreSQL's logical decoding, and transaction logs from SQL Server.

These change events such as inserts, updates, and deletes are then processed by Datastream and transformed into structured formats like AVRO or JSON, ready for storage in Google Cloud, typically in BigQuery tables, enabling near real-time data replication for analytics and other use cases.

Datastream uses unified types to map source to destination data types



Google Cloud

Datastream simplifies data replication by using unified data types to map between different source and destination databases.

This means that regardless of whether your source data is in Oracle as number, MySQL as decimal, PostgreSQL as numeric, or SQL Server as decimal, Datastream will consistently represent it as decimal during replication.

When this data lands in Google Cloud, it can be further transformed into format-specific data types in different file types or destinations, such as Avro as decimal, JSON as number, or stored natively in BigQuery tables as numeric.

This ensures data type consistency and compatibility across different database systems, streamlining the data replication process.

References:

<https://cloud.google.com/datastream/docs/unified-types>

<https://cloud.google.com/datastream/docs/destination-bigquery#map-data-types>

Compare migration and replication options

	gcloud storage	Storage Transfer Service	Transfer Appliance	Datastream
Transfer type	online	online	offline	online
Source locations	On-premises, Google Cloud	On-premises, Google Cloud, multicloud	On-premises	On-premises, Google Cloud, multicloud
Data range	less than 1 TB recommended	more than 1 TB recommended	7 TB, 40 TB, 300 TB	10,000 tables per stream
Velocity	Batch	Batch (hourly at minimum)	Batch	Batch and Stream
Data format	any	any	any	structured

Google Cloud

In summary, Google Cloud offers several data migration and replication options. The “gcloud storage” command is suitable for smaller, online transfers. Storage Transfer Service handles larger online transfers efficiently. Transfer Appliance is ideal for massive offline data migrations. And Datastream provides continuous, online replication of structured data, supporting both batch and streaming velocities.

Choose the option that best fits your data size, transfer type, and data availability requirements.

References:

gcloud storage vs. Storage Transfer service limits:

https://cloud.google.com/storage-transfer/docs/transfer-options#move_data_between_buckets

Storage transfer service known limitations (no sub-hourly support):

<https://cloud.google.com/storage-transfer/docs/known-limitations-transfer#real-time>

Transfer appliance capacities:

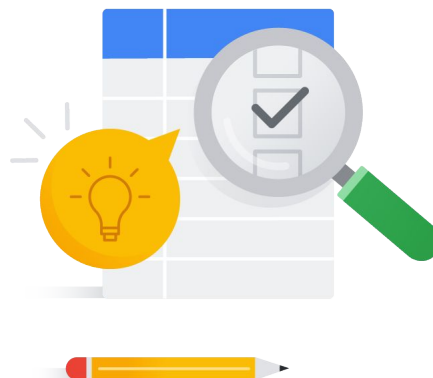
<https://cloud.google.com/transfer-appliance/docs/4.0/specifications#ta-weights>

Lab (optional): Datastream: PostgreSQL Replication to BigQuery

🕒 45 min

Learning objectives

- Prepare a Cloud SQL for PostgreSQL instance using the Google Cloud console.
- Import data into the Cloud SQL instance.
- Create a Datastream connection profile for the PostgreSQL database.
- Create a Datastream connection profile for the BigQuery destination.
- Create a Datastream stream and start replication.
- Validate that the existing data and changes are replicated correctly into BigQuery.



Google Cloud

In this lab, you use Datastream to replicate data from PostgreSQL to BigQuery. You prepare and load a Cloud SQL for PostgreSQL instance. You create Datastream connection profiles for the source and target. You then create a Datastream processing stream and start replication. Finally, you validate the replication in BigQuery.

Lab URL: <https://www.skills.google/focuses/53927?parent=catalog>

MODULE 3

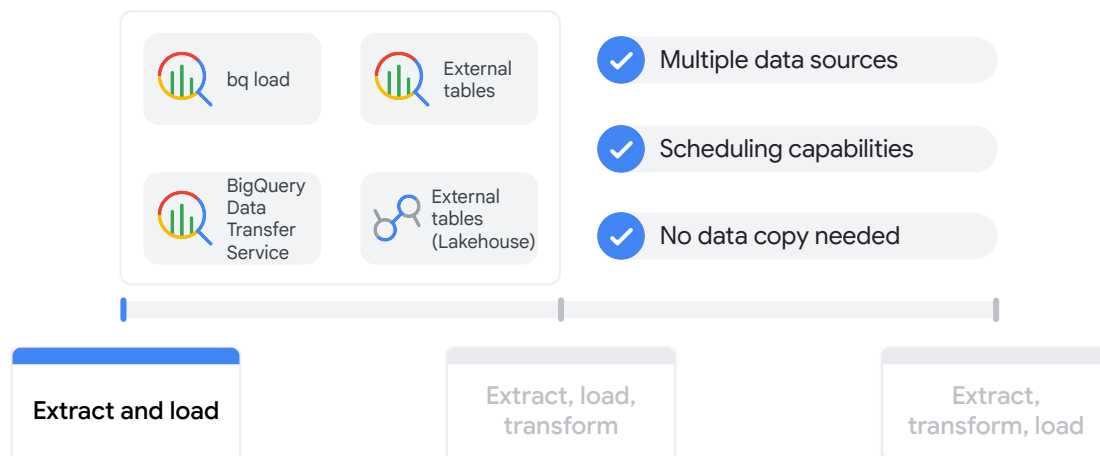
Extract and Load — Data Pipeline Pattern

· 13:15–13:45 ·

slides 4–6, 8, 10, 12–18, 19 (demo), 20

source deck: M3__T-DEGC-B__Extract_and_Load_-_Data_Pipeline_Pattern_ILT_Notes.pdf

Extract and load ingests data into BigQuery without the need for transformation



Google Cloud

The extract and load data pipeline pattern focuses on the tools and options to bring data into BigQuery by eliminating the need for upfront transformation.

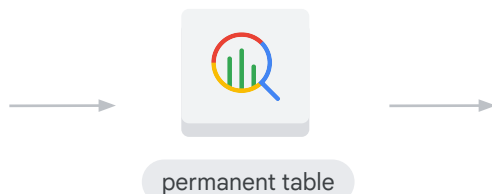
Extract and load greatly simplifies data ingestion into BigQuery. Extract and load leverages tools like `bq load` and BigQuery Data Transfer Service to directly load data from various sources or uses external tables and Lakehouse tables to make data accessible via BigQuery.

This pattern also offers scheduling capabilities and eliminates the need for data copying, promoting efficiency in data pipelines.

BigQuery supports importing from and exporting to multiple formats

Loading data

- Avro
- Parquet
- ORC
- CSV
- JSON
- Firestore export



Query results

- CSV
- JSON
- Google Sheets

Table export

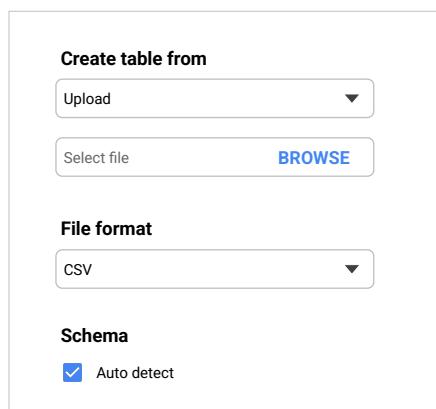
- CSV
- JSON
- Avro
- Parquet

BigQuery provides extensive flexibility in data handling.

It supports loading data from various formats like Avro, Parquet, ORC, CSV, JSON, as well as Firestore exports.

Similarly, you can export BigQuery artifacts, including query results and table data, into formats like CSV, JSON, Avro, and Parquet, facilitating easy integration with other tools and systems.

Load data into BigQuery by using the UI or the **LOAD DATA** statement



Create table from

Upload ▼

Select file **BROWSE**

File format

CSV ▼

Schema

☒ Auto detect

```
# create a new table by loading data from
# CSV files using schema auto-detect
LOAD DATA INTO dataset_name.table_name
FROM FILES(
  format='CSV',
  uris = ['gs://mybucket/*.csv']
)
```



Use **LOAD DATA INTO** to append to an existing table.



Use **LOAD DATA OVERWRITE** to overwrite the existing table.

BigQuery offers two ways to load data: through its friendly user interface for file uploads or via the **LOAD DATA** SQL statement.

The UI simplifies the process, allowing you to select files, specify formats, and even auto-detect schema.

LOAD DATA provides more control, ideal for automation and appending or overwriting existing tables.

References:

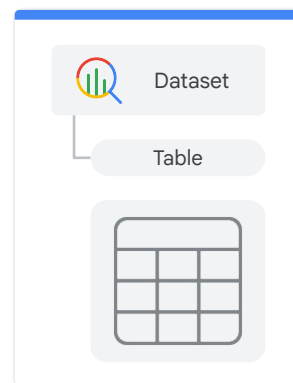
https://cloud.google.com/bigquery/docs/loading-data-cloud-storage-csv#loading_csv_data_into_a_table

https://cloud.google.com/bigquery/docs/reference/standard-sql/other-statements#load_data_statement

Interact with BigQuery by using the `bq` command

```
> bq mk --location=US -dataset dataset-name
```

```
> bq load \
  --source_format=CSV \
  --skip_leading_rows=2 \
  dataset-name.table_name \
  "gs://mybucket/00/*.csv", "gs://mybucket/01/*.csv" \
  ./table_schema.json
```



The Google Cloud SDK `bq` command offers a programmatic way to interact with BigQuery.

You can create BigQuery objects like datasets and tables with the familiar Linux-like command `bq mk`.

The `bq load` command efficiently loads data into BigQuery tables.

Key parameters for `bq load` include specifying the source format such as CSV, skipping header rows, and defining the target dataset and table.

You can load data from multiple files in Cloud Storage using wildcards and optionally provide a schema file for the table structure.

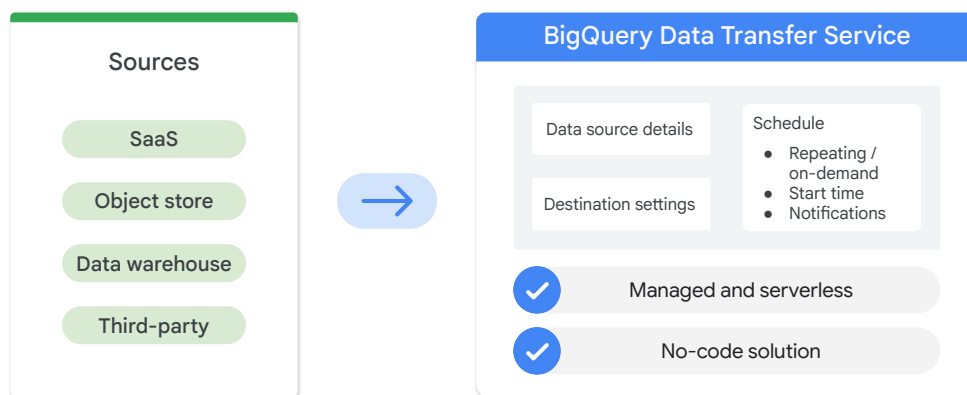
These options provide flexibility and control for loading data into BigQuery from various sources.

References:

<https://cloud.google.com/bigquery/docs/datasets#bq>

<https://cloud.google.com/bigquery/docs/loading-data-cloud-storage-csv#bq>

Use the BigQuery Data Transfer Service to load data from other structured data sources



The BigQuery Data Transfer Service enables seamless loading of structured data from diverse sources, like SaaS applications, object stores, and other data warehouses into BigQuery.

The service provides scheduling options for recurring or on-demand transfers, along with configuration options for data source details and destination settings.

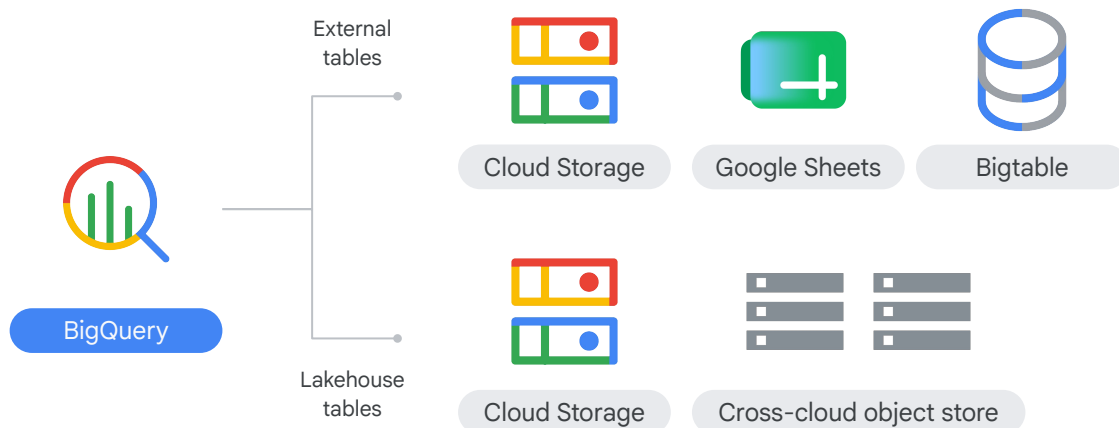
It is a managed and serverless solution, eliminating infrastructure management overhead.

In addition, its no-code approach simplifies data transfer setup and management.

References:

<https://cloud.google.com/bigquery/docs/dts-locations>

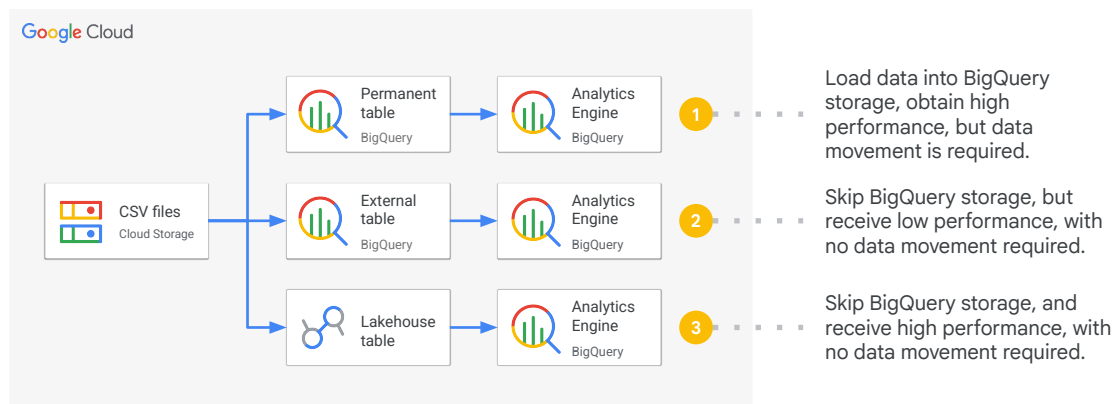
Your data isn't stored in BigQuery? You can still query it!



BigQuery's data access capabilities extend beyond its own storage. BigQuery allows you to query data residing in sources like Cloud Storage, Google Sheets, and Bigtable using external tables.

Additionally, Lakehouse tables provide a way to query data across Cloud Storage and even other cloud object stores, expanding BigQuery's reach and flexibility for data analysis.

Three ways of analyzing structured data in BigQuery



BigQuery offers flexibility in analyzing structured data.

You can load data into permanent BigQuery tables for high-performance analytics, but with data movement involved.

External tables allow you to query data directly in Cloud Storage without loading it into BigQuery, which is suitable for less frequent access.

Lakehouse tables provide the best of both worlds: high-performance analytics on data in Cloud Storage without the need to load it into BigQuery, and without data movement.

External tables let you query Google Sheets data directly in BigQuery

SCHEMA

DETAILS

LINEAGE

...

Table information

Table ID	your-project-id.sheets.sheets_table
...	

External data configuration

Source URI(s)	https://docs.google.com/spreadsheets/...
Source format	GOOGLE_SHEETS



```
# Query the Google Sheets table
SELECT ...
FROM
  `your-project-id.sheets.sheets_table`;
```

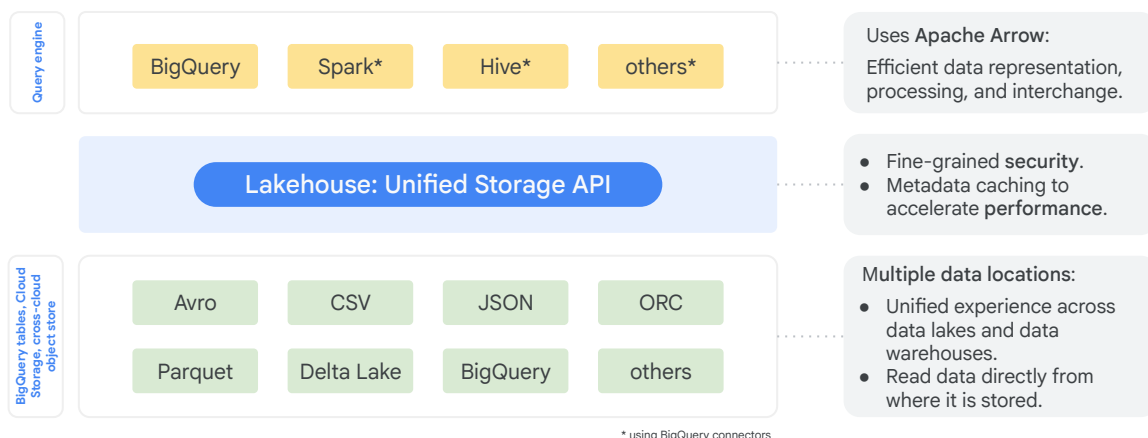
- No query cost estimation, table preview, or query caching available.
- Performance is slower than querying a permanent table in BigQuery.

BigQuery external tables bridge the gap between Google Sheets and BigQuery, enabling direct querying of Sheets data within BigQuery.

By specifying the Google Sheets URL and format, users can treat the sheet as a table in BigQuery, simplifying data analysis across platforms.

However, be aware that querying external tables may have limitations, like slower performance and the unavailability of cost estimation, table preview, and query caching.

Lakehouse: BigQuery on your data lake... and much more!



Google Cloud

Lakehouse extends BigQuery's capabilities, providing a unified interface to query data directly from your data lake and other sources without moving or copying it.

Lakehouse leverages Apache Arrow for efficient data handling and offers fine-grained security and metadata caching.

With Lakehouse, you can seamlessly access data across data lakes and data warehouses using familiar BigQuery tools.

References:

<https://docs.cloud.google.com/bigquery/docs/biglake-intro#limitations>

<https://docs.cloud.google.com/bigquery/docs/biglake-intro#connectors>

A Lakehouse table behaves the same way as a permanent table

SCHEMA

DETAILS

LINEAGE

...

Table information

Table ID	your-project-id.biglake.csv_table
...	

External data configuration

Source URI(s)	gs://your-bucket/your_file.csv
...	



```
# Query the Lakehouse table
SELECT ...
FROM
  `your-project-id.biglake.csv_table`;
```

- ✓ Use Lakehouse if loading data into BigQuery isn't an option for your use case.
- ✓ Queries are performant due to metadata caching.
- No query cost estimation or table preview.

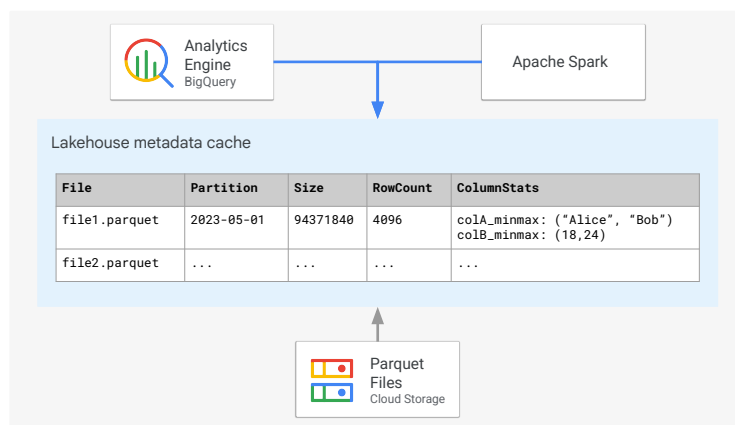
Lakehouse tables provide a seamless querying experience, allowing you to interact with data stored in external sources like Cloud Storage just like you would with data in native BigQuery tables.

You can use standard SQL queries to access and analyze the data within Lakehouse tables, including **SELECT** statements and joins.

Behind the scenes, Lakehouse leverages metadata caching to enhance query performance, even though the data physically resides outside BigQuery.

However, some features like query cost estimation and table preview are not available for Lakehouse tables due to the external nature of the data.

Lakehouse uses metadata caching to accelerate performance



Advantages of BigQuery

- Skip object list
- Faster file and partition pruning
- Dynamic predicate pruning

Advantages of Spark

- Can read metadata statistics using the spark-bigquery connector.

Cache properties

- Staleness between 30 min and 7 days
- Automatic or manual refresh

Google Cloud

Lakehouse maintains a metadata cache. The cache stores details about external data. For example, it can contain details about Parquet files stored in Cloud Storage, such as file size, row count, and column statistics like minimum/maximum values.

This cache allows querying via BigQuery to skip listing all objects, prune files, and partitions faster, and enable dynamic predicate pushdown, resulting in improved query performance.

The cache allows querying by Spark to access metadata statistics that the Spark-BigQuery connector can leverage to speed up queries.

The metadata cache has configurable staleness from 30 minutes to 7 days and it can be refreshed automatically or manually.

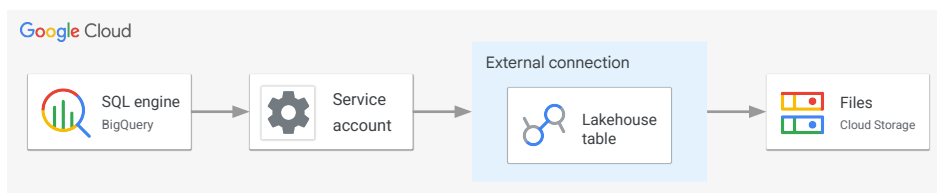
References:

<https://cloud.google.com/blog/products/data-analytics/deep-dive-on-how-biglake-accelerates-query-performance>
<https://cloud.google.com/bigquery/docs/metadata-caching>

Compare security approaches: Non-native stored data



External tables: the user needs separate permissions to access the table and the data source.



Lakehouse: access is delegated using a service account. It decouples access to the table from the underlying data source.

External tables in BigQuery require users to have separate permissions for both the table itself and the underlying data source. This can lead to more complex access management.

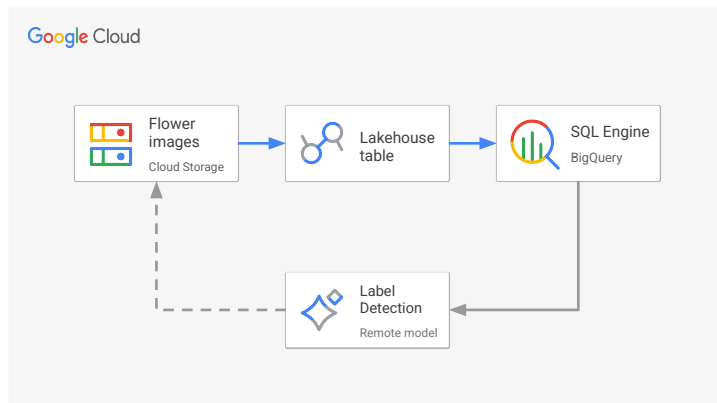
Lakehouse tables offer a streamlined approach. Access is delegated through a service account, decoupling table access from the data source. This simplifies permission management and enhances security.

References:

https://cloud.google.com/bigquery/docs/biglake-intro#security_model

https://cloud.google.com/bigquery/docs/create-cloud-storage-table-biglake#set_up_access_control_policies

[Demo] Analyze an object table by using a remote model



```
gs://biglake-flowers/flower_photos/  
daisy/pic.jpg
```

```
"label_annotations": [{  
  "description": "Flower",  
  "score": 0.98062  
}]
```

Compare external and Lakehouse tables

	External tables	Lakehouse tables
Query Engine	BigQuery	BigQuery, others using BigQuery connector
Data types	• Parquet, Avro, JSON, ORC, CSV • Datastore export, Firestore export • Google Sheets	• Parquet, Avro, JSON, ORC, CSV • Apache Iceberg, Delta, Hudi • Hive partitioned • Any unstructured (via object tables)
Locations	• Cloud Storage • Bigtable • Google Drive	• Cloud Storage • Amazon S3 • Azure Blob Storage
Access pattern	Direct link to the source. The user needs to have access to the source	External connection with access delegation. The service account has access to the source.
Security	None	• Column-level security, data masking • Row-level security

Google Cloud

In summary, both external and Lakehouse tables enable querying data residing outside of BigQuery, but Lakehouse offers broader capabilities.

Lakehouse supports a wider range of data formats and storage locations, including object stores across multiple cloud providers, and provides advanced security features like column-level and row-level security.

External tables are simpler to set up, but lack fine-grained security controls.

Lakehouse tables offer enhanced performance, security, and flexibility for querying external data, making them suitable for enterprise data lake use cases.

References:

https://cloud.google.com/bigquery/docs/external-data-sources#external_data_source_feature_comparison

MODULE 4

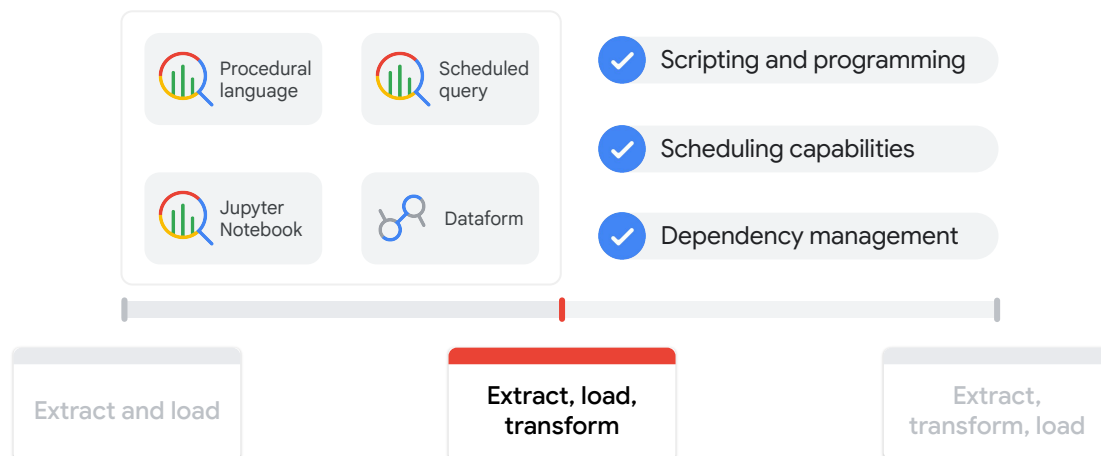
Extract, Load, and Transform (ELT) — Data Pipeline Pattern

· 14:45–15:10 ·

slides 4-5, 7-9, 13, 16-19, 26, 28 (lab intro)

source deck: M4__T-DEGC-B__Extract_Load_and_Transform_-_Data_Pipeline_Pattern_ILT_Notes.pdf

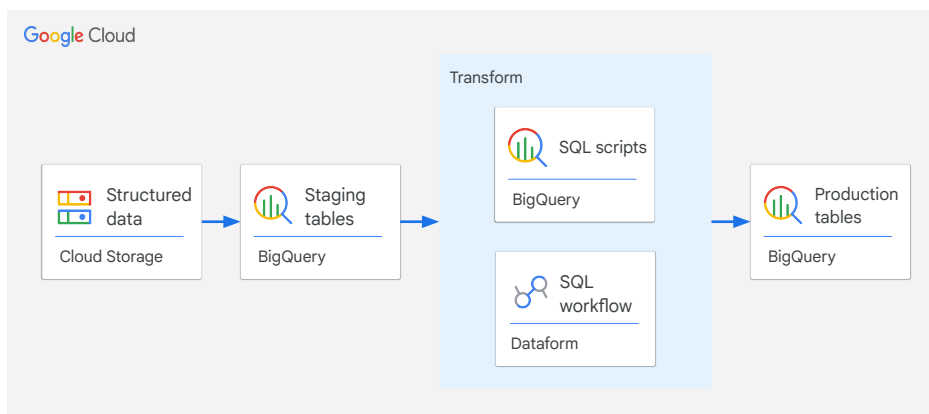
Once data is loaded into BigQuery, there are multiple ways to transform it



Google Cloud

Extract, load, and transform centers around data being loaded into BigQuery first. Once data is loaded, there are multiple ways to transform it. Procedural languages like SQL can be used to transform data. Scheduled queries can be used to transform data on a regular basis. Scripting and programming languages like Python can be used to transform data. And a tool like Dataform simplifies transformation beyond basic programming options.

Use an ELT pipeline to apply transformations after data is staged in BigQuery



In an extract, load, and transform pattern pipeline, structured data is first loaded into BigQuery staging tables. Transformations are then applied within BigQuery itself using SQL scripts or tools like Dataform with SQL workflows. The transformed data is finally moved to production tables in BigQuery, ready for use. This approach leverages BigQuery's processing power for efficient data transformation.

BigQuery supports procedural language queries to perform multiple statements in a sequence

```
-- create variables
DECLARE book_name STRING DEFAULT 'Ulysses';
DECLARE book_year INT64 DEFAULT 1922;

-- Create a temporary table called Books.
EXECUTE IMMEDIATE
  "CREATE TEMP TABLE Books
    (title STRING, publish_date INT64)";

-- add a row for Ulysses, using the variables
-- declared and the ? placeholder
EXECUTE IMMEDIATE
  "INSERT INTO Books (title, publish_date) VALUES(?, ?)"
  USING book_name, book_year;
```

Procedural language in BigQuery:

- Runs multiple statements in a sequence with shared state.
- Automates management tasks, such as creating or dropping tables.
- Implements complex logic using programming constructs like **IF** and **WHILE**.
- Can declare user-created variables or reference BigQuery's system variables.
- Can use transactions (**COMMIT**, **ROLLBACK**).

Google Cloud

With support for procedural language, BigQuery allows the execution of multiple SQL statements in sequence with shared state. This enables the automation of tasks like table creation, implementation of complex logic using constructs like **IF** and **WHILE**, and the use of transactions for data integrity. You can also declare variables and reference system variables within your procedural code.

References:

<https://cloud.google.com/bigquery/docs/reference/standard-sql/procedural-language>
<https://cloud.google.com/bigquery/docs/multi-statement-queries>

User-defined functions (UDFs) transform data by using SQL or JavaScript

```
# create a permanent user-defined function
CREATE FUNCTION dataset_name.AddFourAndDivide
(x INT64, y INT64) RETURNS FLOAT64 AS (
  (x + 4) / y
);
```

```
# use the function in your SQL script
SELECT
  val, dataset_name.AddFourAndDivide(val, 2)
FROM ...;
```

Usage notes:

- Can be persistent (CREATE FUNCTION) or temporary (CREATE TEMPORARY FUNCTION).
- Use SQL where possible. Use JavaScript if the same cannot be accomplished with SQL.
- JavaScript UDFs can use additional input libraries.
- The `bigquery-utils` project and the `bigquery-public-data.persistent_udfs` public dataset contain community-contributed UDFs.

BigQuery supports user-defined functions or UDFs for custom data transformations using SQL or JavaScript. These UDFs can be persistent or temporary, and it is recommended to use SQL UDFs when possible. JavaScript UDFs offer the flexibility to use external libraries, and community-contributed UDFs are available for reuse.

References:

<https://cloud.google.com/bigquery/docs/user-defined-functions>

Stored procedures execute a collection of SQL statements

```
# create a stored procedure
CREATE OR REPLACE PROCEDURE
  dataset_name.create_customer()
BEGIN
  DECLARE id STRING;
  SET id = GENERATE_UUID();
  INSERT INTO dataset_name.customers (customer_id)
    VALUES(id);
  SELECT FORMAT("Created customer %s", id);
END
```

```
# call the procedure in your SQL script
CALL dataset_name.create_customer();
```

Usage notes:

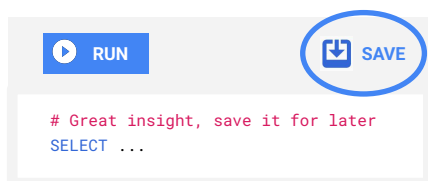
- Can be called from queries or other stored procedures.
- Accept input values and return values as output.
- Can access or modify data across multiple datasets.
- Can contain multi-statement queries and support procedural language statements.

Stored procedures are pre-compiled SQL statement collections, streamlining database operations by encapsulating complex logic for enhanced performance and maintainability. Benefits include reusability, parameterization for flexible input, and transaction handling. Stored procedures are called from applications or within SQL scripts, promoting modular design.

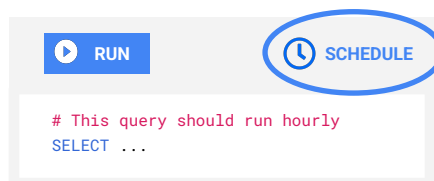
References:

<https://cloud.google.com/bigquery/docs/procedures>

Save your queries and schedule them for repeating use cases



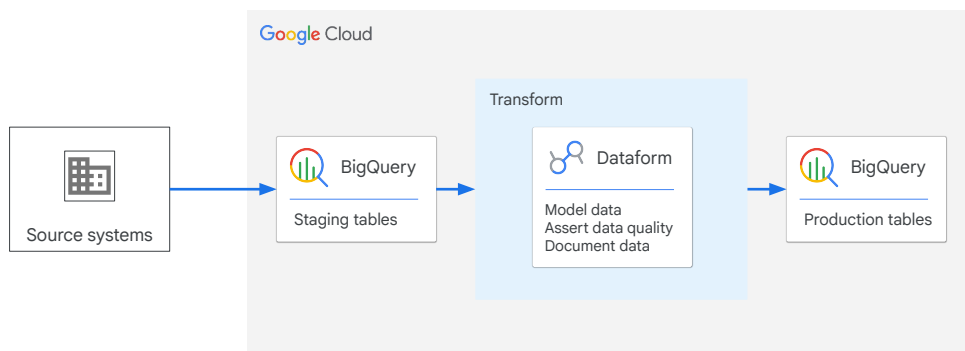
- Is visible in Explorer > Queries.
- Supports version control.
- Share with users or groups.
- Download as `.sql` file.
- Upload other queries from `.sql` files.



- Set repeat frequency.
- Set start and end times.
- Specify destination for query results.
- Can use runtime parameters.
- **Tip:** use Dataform for SQL workflows.

BigQuery offers the option to save and schedule queries for repeated use. You can save queries, manage versions, and share them with others. Scheduling allows you to automate query execution by setting frequency, start and end times, and result destinations. Dataform is recommended for more complex SQL workflows.

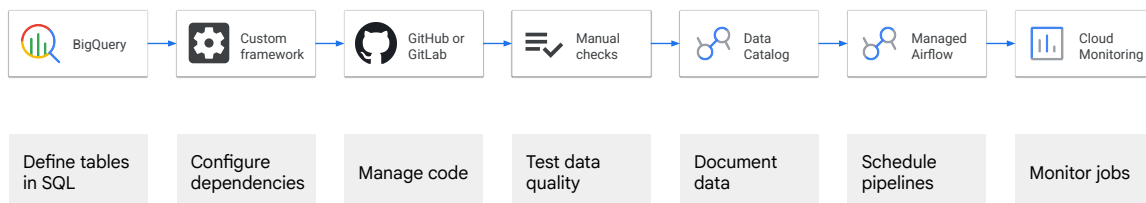
Dataform is a serverless framework to develop and operationalize ELT pipelines in SQL



Dataform is a serverless framework that simplifies the development and management of ELT pipelines using SQL. Dataform enables data transformation within BigQuery, ensuring data quality and providing documentation. This approach streamlines the process of moving data from source systems to production tables in BigQuery, making operations more efficient and manageable.

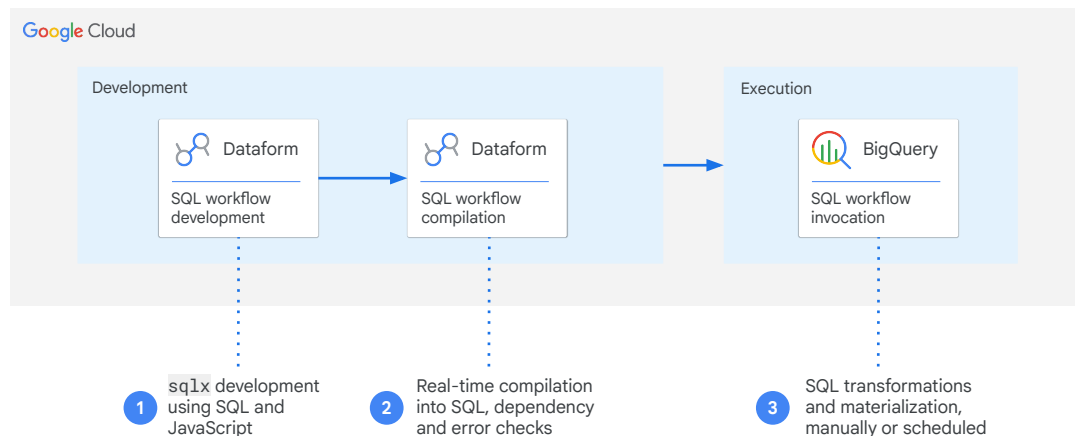
Dataform unifies data transformation, assertion, and automation in BigQuery

Without Dataform, this would be a time-consuming and error-prone process



Dataform streamlines data operations in BigQuery by unifying transformation, assertion, and automation. Without Dataform, tasks like defining tables, managing code, testing data quality, and scheduling pipelines would be time-consuming and prone to errors. They could also involve multiple tools and manual processes. Dataform simplifies these tasks within BigQuery, improving efficiency and data reliability.

SQL workflow development and compilation run in Dataform, workflow execution runs in BigQuery



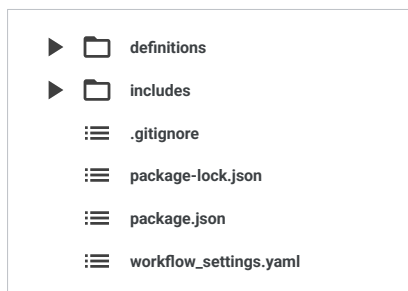
Google Cloud

Dataform and BigQuery work together to manage SQL workflows. With Dataform, developers create and compile SQL workflows using SQL and JavaScript. Dataform then performs real-time compilation, including dependency checks and error handling. Finally, the compiled SQL workflows are executed within BigQuery, enabling SQL transformations and materialization either on-demand or through scheduled runs.

References:

- SQL workflow compilation result:
<https://cloud.google.com/dataform/reference/rest/v1beta1/projects.locations.repositories.compilationResults>
- SQL workflow invocation:
<https://cloud.google.com/dataform/reference/rest/v1beta1/projects.locations.repositories.workflowInvocations>

Development happens in workspaces with default files and folders

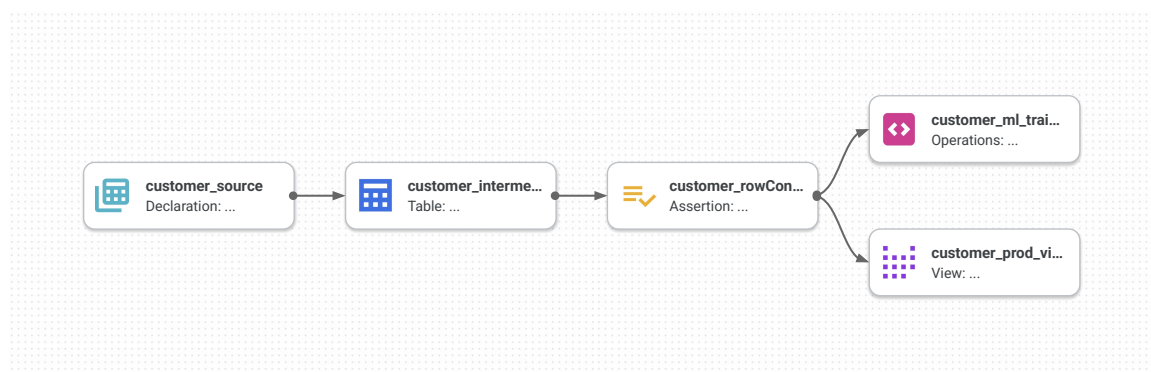


You can add custom files like
README .md as well.

- **definitions folder:**
contains `sqlx` files.
- **includes folder:**
contains JavaScript files.
- **.gitignore file:**
lists files and directories to ignore during Git commits.
- **package.json file (optional):**
defines JavaScript packages and required versions.
- **package-lock.json file (optional):**
contains installed packages with exact versions.
- **workflow_settings.yaml file:**
contains basic settings required to compile the project.

Development using Dataform utilizes workspaces containing default files and folders. Key folders include "definitions" for SQLX files and "includes" for JavaScript files. The ".gitignore" file is used for managing Git commits. Developers may also use "package.json" and "package-lock.json" for handling JavaScript dependencies. The "workflow_settings.yaml" file stores project compilation settings, and custom files like "README.md" can also be added.

The compiled graph visualizes the SQL workflow with its definitions and dependencies



Dataform SQL workflows are best visualized in graph format. The sample workflow starts with a declaration of `customer_source` followed by a `customer_intermediate` table, likely derived from a source system as a pre-processed data source. Next, `customer_rowConsistency` applies assertions for data quality checks.

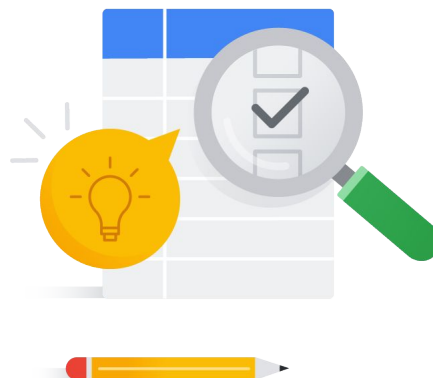
The graph then splits into two paths. In one path, an operation named `customer_ml_training` is invoked. It performs operations on the validated data. In the other path, a view named `customer_prod_view` is created.

Lab: Create and Execute a SQL Workflow in Dataform

🕒 30 min

Learning objectives

- Create a Dataform repository.
- Create and initialize a Dataform development workspace.
- Create and execute a SQL workflow.
- View execution logs in Dataform.



Google Cloud

In this lab, you use Dataform to create and execute a SQL workflow. First, you create a Dataform repository. Second, you create and initialize a Dataform development workspace. Then, you create and execute a SQL workflow. Finally, you view execution logs in Dataform to confirm completion.

Lab URL: <https://www.skills.google/focuses/77087?parent=catalog>

MODULES 5 + 6

Extract, Transform, Load & Automation Techniques (compressed)

· 16:05–16:20 ·

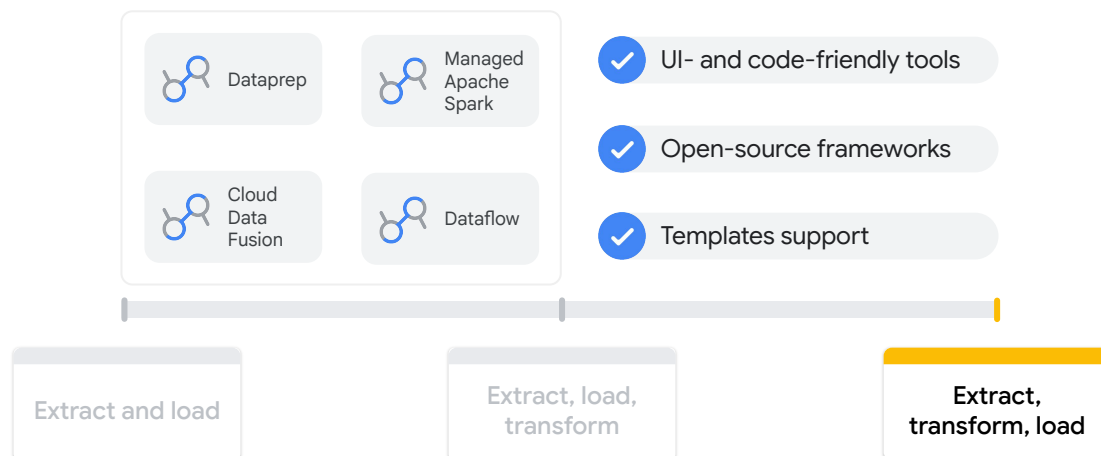
M5 slides 4, 22–27

source deck: M5__T-DEGC-B__Extract_Transform_and_Load_-_Data_Pipeline_Pattern_ILT_Notes.pdf

M6 slides 4–8, 19

source deck: M6__T-DEGC-B__Automation_Techniques_ILT_Notes.pdf

Google Cloud provides multiple services for distributed data processing



Google Cloud

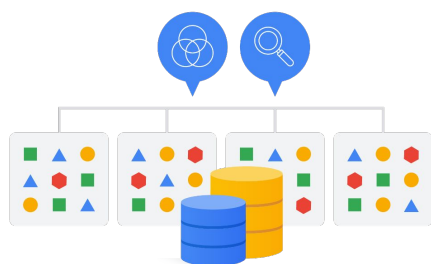
The extract, transform, and load data pipeline pattern focuses on data being adjusted or transformed prior to being loaded into BigQuery. Google Cloud offers a variety of services to handle distributed data processing.

For developers who prefer visual interfaces, there are user-friendly tools like Dataprep and Cloud Data Fusion.

If you favor open-source frameworks, Managed Service for Apache Spark and Dataflow are an option.

And to streamline your workflows, template support is provided across various stages of data processing, from extraction and loading to full-fledged transformation.

Batch processing versus streaming data processing

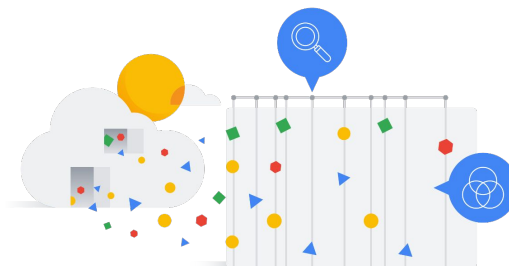


Batch processing

Processing and analysis happens on a set of stored data.

Payroll system

Billing system



Streaming data processing

A flow of data records generated by various data sources.

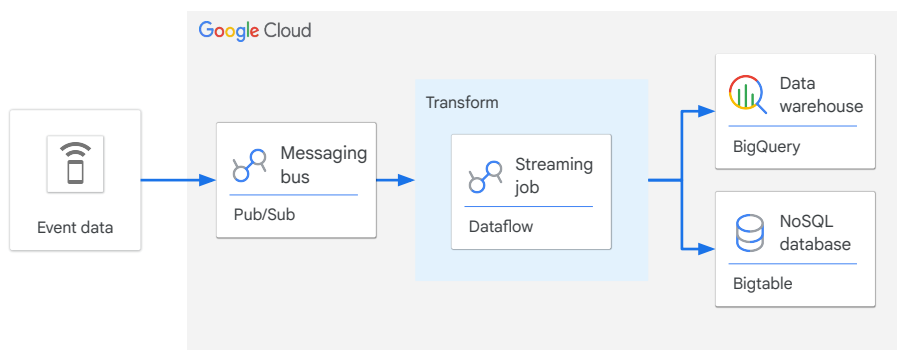
Fraud detection

Intrusion detection

Batch processing involves analyzing a fixed set of stored data, suitable for tasks like payroll or billing systems.

On the other hand, streaming data processing handles a continuous flow of data from various sources, making it ideal for real-time applications like fraud or intrusion detection.

Streaming ETL workloads require continuous data ingestion, processing, and near real-time analytics

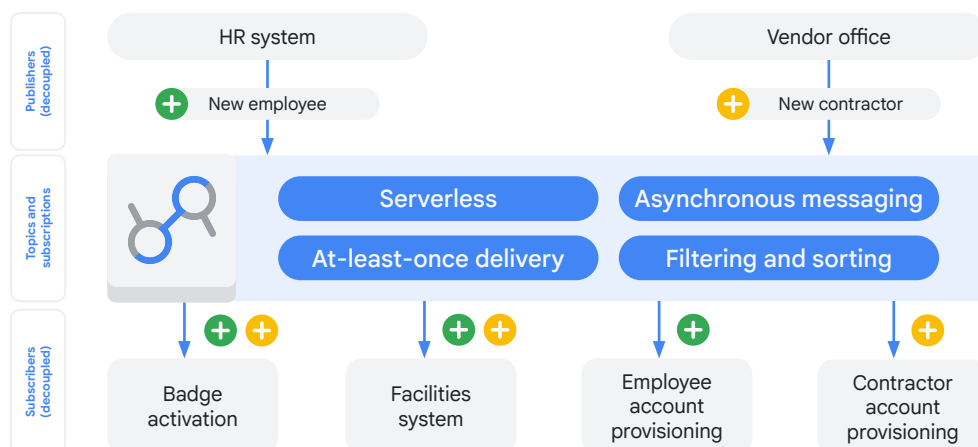


Streaming ETL workloads on Google Cloud involve the continuous ingestion of event data, often through Pub/Sub.

This data is often processed in real-time using Dataflow, allowing for transformations and enrichment.

Finally, the processed data is loaded into various destinations like BigQuery for analytics, enabling near real-time insights, or Bigtable for NoSQL storage.

Use Pub/Sub to ingest high volumes of event data and distribute to different consuming systems



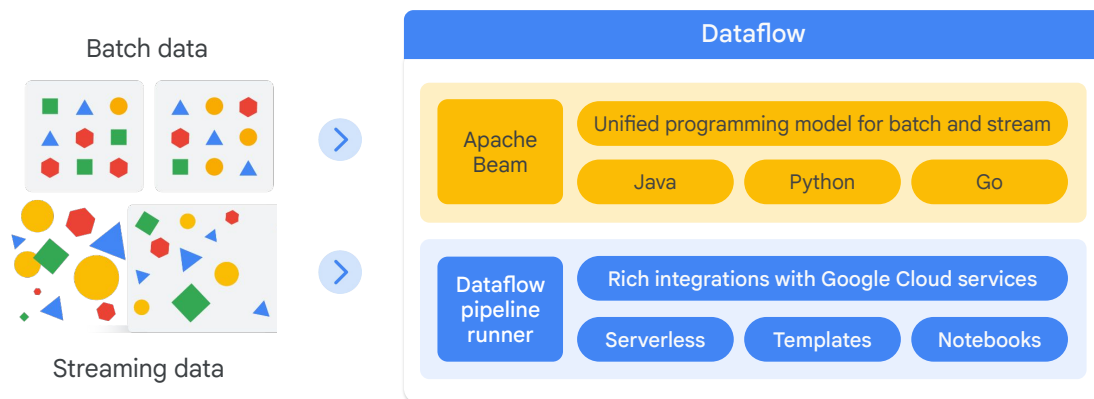
Google Cloud

Pub/Sub can efficiently manage high volumes of event data.

Pub/Sub acts as a central hub, receiving events like "New employee" or "New contractor" from various sources.

Pub/Sub then distributes these events to relevant systems like badge activation, facilities, and account provisioning, ensuring reliable delivery and enabling decoupled, asynchronous communication between systems.

Dataflow can process both batch and streaming data using the Apache Beam programming model



Dataflow leverages the Apache Beam programming framework to efficiently process both batch and streaming data.

This unified approach simplifies development, allowing you to use languages like Java, Python, or Go.

Dataflow seamlessly integrates with other Google Cloud services and offers features like a pipeline runner, serverless execution, templates, and notebooks for a streamlined experience.

Example: Streaming and transforming messages from Pub/Sub to BigQuery

```
with beam.Pipeline(options=pipeline_options) as pipeline:

    # Read messages from Pub/Sub
    messages = pipeline | 'read' >> ReadFromPubSub(topic='<PUBSUB_TOPIC>')

    # Parse messages
    def parse_message(message):
        return json.loads(message.decode('utf-8'))

    parsed_messages = messages | 'parse' >> beam.Map(parse_message)

    # Write parsed messages to BigQuery
    parsed_messages | 'write' >> WriteToBigQuery(
        table='<BIGQUERY_TABLE>',
        schema=table_schema,
        create_disposition=BigQueryDisposition.CREATE_IF_NEEDED,
        write_disposition=BigQueryDisposition.WRITE_APPEND
    )

# Run the pipeline
pipeline.run()
```

ReadFromPubSub():

- Retrieves messages from a Pub/Sub topic or subscription.

beam.Map():

- Applies a specified transformation to the message.

WriteToBigQuery():

- Writes the transformed message into a BigQuery table.

Google Cloud

This code example demonstrates how to use Apache Beam to stream messages from Pub/Sub, transform them using a parsing function, and then write the results into BigQuery.

The “**ReadFromPubSub**” function retrieves messages, “**Beam.Map()**” applies the parsing transformation, and “**WriteToBigQuery**” loads the transformed data into a specified BigQuery table, creating the table if necessary and appending new data to it.

Use Dataflow templates for recurring pipeline executions with different parameters

+ CREATE JOB FROM TEMPLATE

Dataflow template
Cloud Storage Text to BigQuery (Stream) ▼

Source
gs:// Cloud Storage Input File(s) **BROWSE**

Target
BigQuery output table **BROWSE**

Required Parameters ▼

Benefits of using templates

- Pipeline design is separate from deployment.
- Parameters customize the pipeline and make it reusable with different inputs.
- Templated pipelines can be deployed from the Google Cloud console, command-line interface, or API.
- Use Google-provided pre-built templates for common scenarios.



You can create your own custom templates.

Google Cloud

Dataflow templates allow you to create reusable pipelines for recurring tasks.

You can separate the pipeline design from its deployment, making it easier to manage and update.

By using parameters, you can customize the pipeline for different inputs, increasing its versatility.

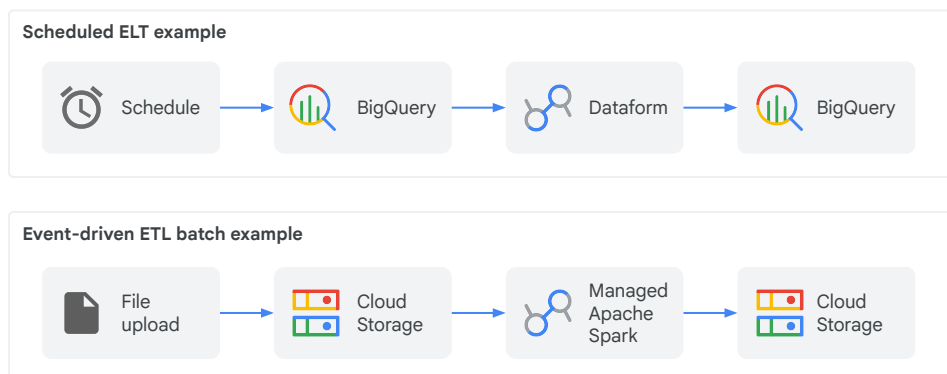
These templated pipelines can be easily deployed through various methods, and Google provides pre-built templates for common scenarios.

References:

<https://cloud.google.com/dataflow/docs/concepts/dataflow-templates>

<https://cloud.google.com/dataflow/docs/guides/templates/provided/text-to-bigquery-stream> (used in lab)

ELT and ETL workloads can be automated to run on a recurring basis

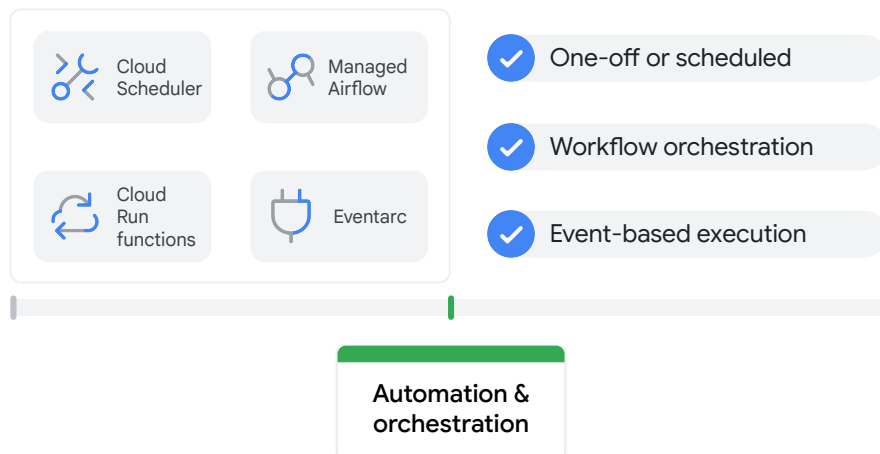


On Google Cloud, ELT and ETL workloads can be automated for recurring execution.

For example, in a scheduled ELT, a defined schedule triggers data extraction from BigQuery, transformation via Dataform, and loading back into BigQuery.

Meanwhile, in the example shown for an event-driven ETL, a file upload to Cloud Storage initiates a batch process using Managed Service for Apache Spark, culminating in data landing in Cloud Storage.

Google Cloud provides multiple services for automating and orchestrating your workloads



Google Cloud offers a suite of services to automate and orchestrate your workloads.

For scheduled tasks or one-off jobs, you can leverage Cloud Scheduler and Managed Airflow.

If your workflows require orchestration, Managed Airflow is the ideal choice.

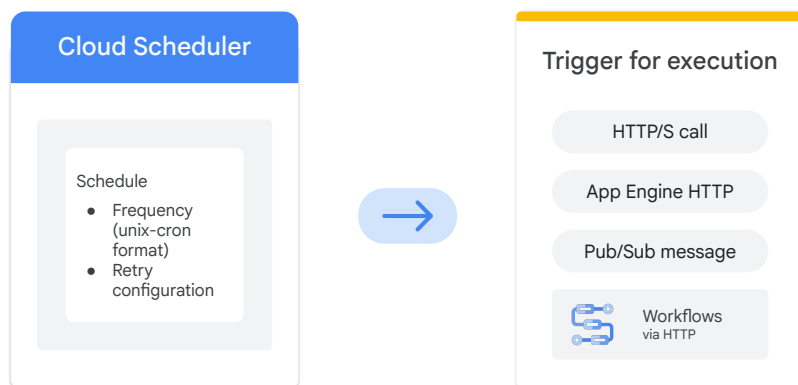
To trigger actions based on events, consider using Cloud Run functions or Eventarc.

Automation techniques

- | | |
|----|---|
| 01 | Automation patterns and options for pipelines |
| 02 | Cloud Scheduler and Workflows |
| 03 | Managed Service for Apache Airflow |
| 04 | Cloud Run functions |
| 05 | Eventarc |



Cloud Scheduler invokes your workloads at recurring intervals



Cloud Scheduler empowers you to automate tasks by invoking your workloads at specified, recurring intervals.

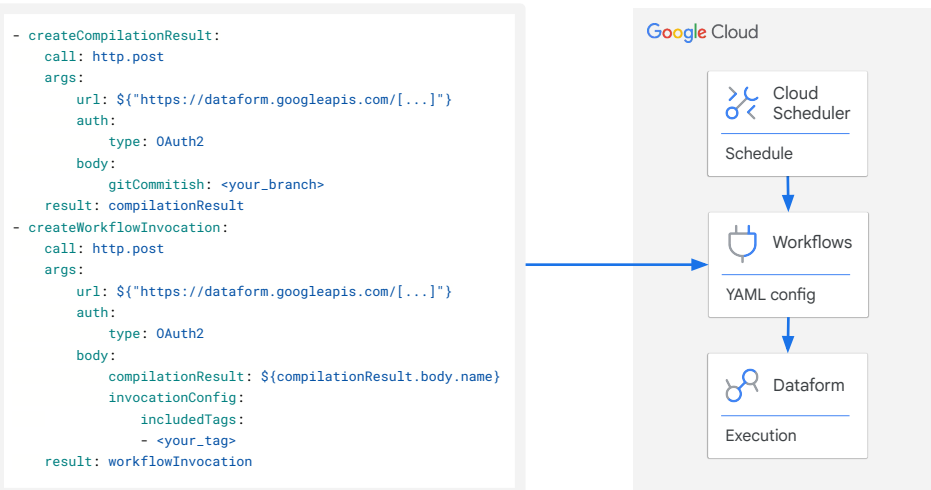
It grants you the flexibility to define both the frequency and precise time of day for job execution.

Triggers can be based on HTTPS calls, App Engine HTTP calls, Pub/Sub messages, or Workflows.

References:

<https://cloud.google.com/scheduler/docs/overview>

Example: Trigger a Dataform SQL workflow



Google Cloud

Cloud Scheduler can be used to trigger a Dataform SQL workflow.

In the example code, a scheduled job in Cloud Scheduler initiates the process, defined in a YAML config file.

The workflow involves two main steps: creating a compilation result from your Dataform code and then triggering a workflow invocation using that result, ensuring only specific parts of your Dataform project execute based on included tags.

References:

<https://cloud.google.com/dataform/docs/schedule-executions-workflows>

Compare automation options

	Cloud Scheduler	Managed Airflow	Cloud Run functions	Eventarc
Trigger type	schedule, manual	schedule, manual	event	event
Serverless	yes	no	yes	yes
Coding effort	low	medium	high	high
Programming languages	YAML (with Workflows)	Python	Python, Java, Go, Node.js, Ruby, PHP, .NET core	any

Google Cloud

In summary, there are various Google Cloud data-related automation options.

Cloud Scheduler and Managed Airflow are suitable for scheduled or manual triggers, while Cloud Run functions and Eventarc are event-driven.

Cloud Scheduler offers low coding effort with YAML, and Managed Airflow requires medium effort with Python.

Cloud Run functions support multiple languages, while Eventarc is language-agnostic.

As a final note, all options except Managed Airflow are serverless.